

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI**

Đỗ Quang Hợp

**NGHIÊN CỨU VÀ XÂY DỰNG NỀN TẢNG
EDGE AI TÍCH HỢP AI AGENT VÀ TỐI ƯU
MÔ HÌNH YOLO**

**OPTIMIZING YOLO FOR EDGE AI AND DEVELOPING AN
EDGE-AGENT MANAGEMENT SYSTEM WITH DASHBOARD
AND AI AGENT**

ĐỒ ÁN TỐT NGHIỆP

Ngành: Khoa Học Máy Tính
Khoa: Khoa Công Nghệ Thông Tin

HÀ NỘI – 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI**

Đỗ Quang Hợp

**NGHIÊN CỨU VÀ XÂY DỰNG NỀN TẢNG
EDGE
AI TÍCH HỢP AI AGENT VÀ TỐI ƯU MÔ
HÌNH
YOLO**

**OPTIMIZING YOLO FOR EDGE AI AND DEVELOPING AN
EDGE-AGENT MANAGEMENT SYSTEM WITH DASHBOARD
AND AI AGENT**

ĐỒ ÁN TỐT NGHIỆP

**Ngành: Khoa Học Máy Tính
Khoa: Khoa Công Nghệ Thông Tin**

**Giảng viên hướng dẫn: TS. Hoàng Nam Thắng
Sinh viên thực hiện: Đỗ Quang Hợp
Mã số sinh viên: 68CNCS
Lớp: 68CS2**

HÀ NỘI – 2026

Trang này dùng để dán bản Nhận xét của người hướng dẫn, hoặc thay bằng bản nhận xét đã ký.

Trang này dùng để dán bản Nhận xét của người phản biện, hoặc thay bằng bản nhận xét đã ký.

TÓM TẮT

Tên đề tài: NGHIÊN CỨU VÀ XÂY DỰNG NỀN TẢNG EDGE AI TÍCH HỢP AI AGENT VÀ TỐI ƯU MÔ HÌNH YOLO

Sinh viên thực hiện: Đỗ Quang Hợp

Mã SV: 68CNCS

Lớp: 68CS2

Đề án nghiên cứu đồng thời hai vấn đề: tối ưu mô hình YOLO cho thiết bị Edge AI và xây dựng nền tảng quản lý vòng đời AI trên một đội thiết bị phân tán. Ở phần mô hình, đề án xây dựng YOLO baseline, khảo sát structured pruning với nhiều tỷ lệ, Quantization-Aware Training và pipeline kết hợp pruning–fine-tuning–QAT. Các checkpoint được chuyển đổi sang ONNX và TensorRT ở các chế độ FP16, INT8, sau đó đánh giá trên Jetson Nano và Jetson Orin Nano theo độ chính xác, latency, FPS, kích thước model, GPU memory, power và nhiệt độ.

Ở phần nền tảng, đề án tổ chức hệ thống theo kiến trúc Cloud Control Plane–Edge Agent–Edge Runtime. Control Plane cung cấp Fleet Management, Model Registry, Deployment, Command, Monitoring, Incident và Knowledge; Edge Agent thực hiện đồng bộ desired state, tải và kiểm tra artifact, điều khiển runtime, thu telemetry và rollback khi triển khai thất bại. AI Copilot kết hợp RAG với tool calling/MCP để truy vấn tài liệu và trạng thái vận hành. Các hành động thay đổi hệ thống phải được kiểm tra quyền, lập kế hoạch, phê duyệt và ghi audit.

Giải pháp được đánh giá bằng một kịch bản xuyên suốt từ huấn luyện, tối ưu, đóng gói và phát hành model đến triển khai trên hai lớp Jetson, theo dõi suy giảm FPS, tạo incident, phân tích bằng Copilot và rollback. Kết quả định lượng cuối cùng được để ở trạng thái cần bổ sung cho đến khi hoàn thành phép đo trên thiết bị thực tế. Đóng góp của đề án là kết nối nghiên cứu tối ưu YOLO với cơ chế vận hành model có phiên bản, có khả năng quan sát, khôi phục và hỗ trợ ra quyết định.

Trang này dùng để dán bản Nhiệm vụ đồ án tốt nghiệp, hoặc thay bằng bản nhiệm vụ đã được phê duyệt.

LỜI NÓI ĐẦU

Sự phát triển nhanh của trí tuệ nhân tạo tại biên đang mở ra khả năng xử lý dữ liệu hình ảnh trực tiếp tại nơi dữ liệu được sinh ra. Cách tiếp cận này giúp giảm độ trễ, hạn chế lưu lượng truyền về máy chủ và tăng khả năng hoạt động khi kết nối mạng không ổn định. Tuy nhiên, việc đưa mô hình phát hiện đối tượng YOLO lên thiết bị có tài nguyên hạn chế không chỉ là bài toán tối ưu mô hình. Một hệ thống thực tế còn cần cơ chế triển khai phiên bản, giám sát trạng thái, thu thập chỉ số, điều khiển từ xa và hỗ trợ người vận hành ra quyết định.

Từ bối cảnh đó, đồ án lựa chọn đề tài “Tối ưu mô hình YOLO cho Edge AI và xây dựng hệ thống quản lý Edge Agent tích hợp Dashboard và AI Agent”. Nội dung báo cáo tập trung vào hai phần có quan hệ chặt chẽ: tối ưu chuỗi suy luận để mô hình đáp ứng ràng buộc của thiết bị biên; đồng thời thiết kế nền tảng quản lý tập trung cho phép theo dõi, cấu hình và điều phối nhiều Edge Agent. AI Agent được đặt trong lớp điều khiển có kiểm soát, nhằm chuyển yêu cầu ngôn ngữ tự nhiên thành kế hoạch và lệnh hệ thống có thể kiểm tra, phê duyệt và truy vết.

Trong quá trình thực hiện đồ án, em đã nhận được sự hướng dẫn và hỗ trợ của nhiều cá nhân, tập thể. Em xin trân trọng cảm ơn TS. Hoàng Nam Thắng đã định hướng chuyên môn, góp ý phương pháp thực hiện và dành thời gian nhận xét các phiên bản của đồ án. Em cảm ơn các thầy cô của Trường Đại học Xây dựng Hà Nội đã trang bị kiến thức nền tảng trong suốt quá trình học tập. Em cũng xin cảm ơn gia đình, bạn bè và các thành viên đã hỗ trợ thử nghiệm hệ thống. Mặc dù đã cố gắng hoàn thiện, báo cáo khó tránh khỏi thiếu sót; em mong nhận được ý kiến đóng góp để tiếp tục cải tiến kết quả nghiên cứu.

Hà Nội, ngày [CÁN BỔ SUNG: ngày] tháng [CÁN BỔ SUNG: tháng] năm 2026

Sinh viên thực hiện

Đỗ Quang Hợp

CAM ĐOAN

Tôi xin cam đoan đồ án tốt nghiệp này là kết quả học tập, nghiên cứu và triển khai của bản thân dưới sự hướng dẫn của TS. Hoàng Nam Thắng. Các nội dung tham khảo, số liệu và kết quả của công trình khác được sử dụng trong báo cáo đều được trích dẫn nguồn rõ ràng. Các hình vẽ, bảng biểu và kết quả thực nghiệm do tôi tạo ra được trình bày trung thực; những dữ liệu chưa được đo kiểm được đánh dấu để bổ sung, không được sử dụng như kết quả đã xác nhận.

Tôi chịu trách nhiệm về tính trung thực, tính nguyên bản và việc tuân thủ quy định liên chính học thuật của toàn bộ nội dung đồ án. Nếu phát hiện có hành vi sao chép, giả mạo số liệu hoặc sử dụng kết quả của người khác mà không trích dẫn phù hợp, tôi xin hoàn toàn chịu trách nhiệm theo quy định của Nhà trường.

Hà Nội, ngày [CẦN BỔ SUNG: ngày] tháng [CẦN BỔ SUNG: tháng] năm 2026

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Đỗ Quang Hợp

Mục lục

LỜI NÓI ĐẦU	i
LỜI CAM ĐOAN	ii
MỤC LỤC	ii
DANH SÁCH HÌNH VẼ	iii
DANH SÁCH BẢNG	iv
DANH SÁCH CÁC KÝ HIỆU, CHỮ VIẾT TẮT	vi
1 TỔNG QUAN ĐỀ TÀI	1
1.1 Đặt vấn đề	1
1.1.1 Sự phát triển của AI tại Edge	1
1.1.2 Hạn chế của triển khai AI trên Edge Device	1
1.1.3 Bài toán quản lý và triển khai AI Model trên nhiều Edge Device	2
1.1.4 Sự cần thiết của tối ưu mô hình AI	2
1.2 Mục tiêu đề tài	3
1.2.1 Mục tiêu tổng quát	3
1.2.2 Mục tiêu cụ thể.....	3
1.3 Phạm vi đề tài	4
1.4 Đối tượng nghiên cứu.....	4
1.5 Phương pháp nghiên cứu.....	4
1.6 Đóng góp của đề tài	5
1.7 Cấu trúc đồ án.....	5
1.8 Kết chương	6

2	CƠ SỞ LÝ THUYẾT	7
2.1	Edge AI.....	7
2.1.1	Khái niệm Edge AI.....	7
2.1.2	Kiến trúc Cloud–Edge	7
2.1.3	Đặc điểm của Edge Device	8
2.1.4	Các vấn đề khi triển khai AI trên Edge.....	8
2.2	Object Detection và YOLO.....	8
2.2.1	Bài toán Object Detection.....	8
2.2.2	Kiến trúc YOLO	9
2.2.3	Quy trình huấn luyện YOLO	9
2.2.4	Các chỉ số đánh giá.....	9
2.3	Model Compression.....	10
2.3.1	Tổng quan Model Compression	10
2.3.2	Model Pruning	10
2.3.3	Structured Pruning	10
2.3.4	Unstructured Pruning	10
2.3.5	Pruning Ratio.....	11
2.3.6	Fine-tuning sau Pruning.....	11
2.4	Model Quantization	11
2.4.1	Khái niệm Quantization.....	11
2.4.2	FP32, FP16 và INT8	11
2.4.3	Post-Training Quantization	12
2.4.4	Quantization-Aware Training	12
2.4.5	Fake Quantization.....	12
2.4.6	So sánh PTQ và QAT	12
2.5	TensorRT	13
2.5.1	Kiến trúc TensorRT	13
2.5.2	ONNX.....	13

2.5.3	TensorRT Engine.....	13
2.5.4	FP16 Inference	13
2.5.5	INT8 Inference	13
2.5.6	TensorRT trên NVIDIA Jetson.....	14
2.6	NVIDIA Jetson.....	14
2.6.1	Jetson Nano	14
2.6.2	Jetson Orin Nano.....	14
2.6.3	So sánh tài nguyên phần cứng	15
2.6.4	CUDA và GPU Acceleration	15
2.7	AI Model Deployment.....	15
2.7.1	Model Registry.....	15
2.7.2	Model Versioning	16
2.7.3	Model Artifact.....	16
2.7.4	Deployment Strategy.....	16
2.7.5	Rollback.....	16
2.8	Edge Runtime	16
2.8.1	AI Inference Pipeline	16
2.8.2	Runtime Lifecycle.....	17
2.8.3	GPU Resource Management.....	17
2.8.4	Runtime Monitoring	17
2.9	Distributed Edge Platform	18
2.9.1	Cloud Control Plane.....	18
2.9.2	Edge Agent.....	18
2.9.3	Telemetry	18
2.9.4	Message Broker	18
2.9.5	Monitoring	18
2.10	RAG và AI Agent.....	19
2.10.1	Retrieval-Augmented Generation.....	19

2.10.2	Embedding Search	19
2.10.3	Hybrid Search	19
2.10.4	AI Agent.....	19
2.10.5	Tool Calling	19
2.10.6	MCP	20
2.11	Domain-Driven Design	20
2.11.1	Entity	20
2.11.2	Value Object.....	20
2.11.3	Aggregate.....	20
2.11.4	Repository	21
2.11.5	Domain Service	21
2.11.6	Bounded Context.....	21
2.12	Kết chương	21
3	NGHIÊN CỨU VÀ TỐI ƯU MÔ HÌNH YOLO	22
3.1	Bài toán tối ưu.....	22
3.1.1	Mô hình baseline	22
3.1.2	Hạn chế của mô hình baseline	22
3.1.3	Mục tiêu tối ưu	22
3.2	Xây dựng mô hình YOLO baseline.....	23
3.2.1	Dataset.....	23
3.2.2	Tiền xử lý dữ liệu	23
3.2.3	Huấn luyện	23
3.2.4	Đánh giá baseline	24
3.3	Tối ưu bằng Pruning	24
3.3.1	Lựa chọn phương pháp pruning	24
3.3.2	Xác định pruning ratio	24
3.3.3	Fine-tuning	24
3.3.4	Đánh giá mô hình sau pruning.....	25

3.4	Tối ưu bằng QAT.....	25
3.4.1	Thiết lập quantization	25
3.4.2	Fake Quantization.....	25
3.4.3	QAT Training.....	25
3.4.4	Export INT8.....	26
3.4.5	Đánh giá mô hình QAT	26
3.5	Kết hợp Pruning và QAT.....	26
3.5.1	Quy trình kết hợp.....	26
3.5.2	Các cấu hình thực nghiệm	27
3.5.3	So sánh kết quả.....	27
3.6	Chuyển đổi sang TensorRT	28
3.6.1	Export ONNX.....	28
3.6.2	TensorRT FP16.....	28
3.6.3	TensorRT INT8.....	28
3.6.4	Kiểm tra độ chính xác sau chuyển đổi.....	28
3.7	Lựa chọn mô hình triển khai.....	28
3.8	Kết chương	29
4	PHÂN TÍCH VÀ THIẾT KẾ NỀN TẢNG EDGE AI	30
4.1	Yêu cầu hệ thống.....	30
4.1.1	Functional Requirements	30
4.1.2	Non-functional Requirements.....	31
4.2	Kiến trúc tổng thể	31
4.3	Thiết kế Domain	32
4.3.1	Fleet Domain	32
4.3.2	Model Registry Domain	32
4.3.3	Deployment Domain.....	33
4.3.4	Command Domain	33
4.3.5	Incident Domain.....	33

4.3.6	Knowledge Domain	33
4.3.7	Ops Domain	33
4.3.8	Copilot.....	33
4.4	Thiết kế Cloud Control Plane	34
4.4.1	Fleet Service.....	34
4.4.2	Model Registry	34
4.4.3	Deployment Service.....	34
4.4.4	Command Service.....	34
4.4.5	Incident Service	34
4.4.6	Knowledge Service	35
4.5	Thiết kế Edge Agent	35
4.5.1	Command Processing	35
4.5.2	Desired State và Actual State	35
4.5.3	Model Lifecycle.....	36
4.5.4	Runtime Lifecycle.....	36
4.5.5	Telemetry	36
4.6	Thiết kế Edge Runtime.....	36
4.7	Thiết kế Model Deployment	37
4.8	Thiết kế Monitoring và Incident	38
4.8.1	Không gian đặc trưng Telemetry (8 Telemetry Features)	38
4.8.2	Kiến trúc phát hiện sự cố đa tầng và Feature Engineering	39
4.8.3	Cơ chế phát hiện suy giảm đồng thời (Pipeline Degradation)	39
4.9	Thiết kế Knowledge và Copilot.....	40
4.10	Kết chương	41
5	XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG	42
5.1	Môi trường triển khai	42
5.1.1	Cloud Server	42

5.1.2	Jetson Nano	42
5.1.3	Jetson Orin Nano	42
5.1.4	Network	42
5.2	Xây dựng Cloud.....	43
5.2.1	Fleet Management.....	43
5.2.2	Model Registry	43
5.2.3	Deployment Management.....	44
5.2.4	Command Management.....	44
5.2.5	Incident Management	44
5.2.6	Knowledge Management	44
5.3	Xây dựng Edge Agent	45
5.3.1	MQTT Communication	45
5.3.2	Command Handler	45
5.3.3	Model Downloader.....	45
5.3.4	Runtime Manager	45
5.3.5	Telemetry Collector	46
5.4	Xây dựng Edge Runtime.....	46
5.4.1	Camera Input	46
5.4.2	YOLO Inference	46
5.4.3	TensorRT	46
5.4.4	Result Processing	46
5.4.5	Runtime Metrics.....	47
5.5	Xây dựng AI Copilot.....	47
5.5.1	Agent Architecture	47
5.5.2	Knowledge Search	47
5.5.3	MCP Tools	47
5.5.4	Tool Calling	47
5.5.5	Conversation Memory	48

5.6	Xây dựng Monitoring	48
5.6.1	Metrics Collection.....	48
5.6.2	Rule-based Detection	48
5.6.3	Incident Creation	48
5.6.4	Alert Processing.....	48
5.7	Giao diện quản lý	49
5.7.1	Dashboard	49
5.7.2	Fleet	49
5.7.3	Models.....	49
5.7.4	Deployments	49
5.7.5	Monitoring	49
5.7.6	Incidents.....	49
5.7.7	Copilot.....	50
5.8	Kết chương	50
6	THỰC NGHIỆM VÀ ĐÁNH GIÁ	51
6.1	Môi trường thực nghiệm	51
6.2	Benchmark mô hình.....	51
6.3	Benchmark trên Jetson Nano	54
6.4	Benchmark trên Jetson Orin Nano.....	56
6.5	So sánh Jetson Nano và Jetson Orin Nano	58
6.6	Benchmark Edge Runtime	58
6.7	Benchmark Platform	59
6.8	Đánh giá khả năng triển khai	60
6.9	Kết chương	61
7	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	62
7.1	Kết quả đạt được	62
7.1.1	Kết quả tối ưu YOLO	62

7.1.2	Kết quả Edge Runtime	62
7.1.3	Kết quả Edge Platform	62
7.1.4	Kết quả AI Copilot	63
7.2	Hạn chế	63
7.3	Hướng phát triển	63
	TÀI LIỆU THAM KHẢO	65
A	CẤU TRÚC SOURCE CODE	67
B	DATABASE SCHEMA	68
C	API SPECIFICATION	69
D	MCP TOOL SPECIFICATION	70
E	MODEL TRAINING CONFIGURATION	71
F	BENCHMARK CONFIGURATION	72
G	DEPLOYMENT CONFIGURATION	73

Danh sách hình vẽ

1.1	Vòng đời AI tại Edge được nghiên cứu trong đồ án	3
2.1	Pipeline suy luận tại Edge	17
3.1	Pipeline kết hợp pruning và QAT	27
4.1	Kiến trúc tổng thể nền tảng Edge AI	32
4.2	Các thành phần chính của Edge Agent	35
4.3	Thiết kế pipeline Edge Runtime	36
4.4	Luồng triển khai model từ Registry đến Runtime	37
4.5	Kiến trúc xử lý đặc trưng Telemetry và phân loại sự cố kết hợp LLM Agent	39
4.6	Luồng truy xuất tri thức của Copilot	40
7.1	Các hướng phát triển từ kết quả hiện tại	64

Danh sách bảng

2.1	So sánh PTQ và QAT	12
2.2	Thông tin phần cứng cần ghi nhận cho hai thiết bị Jetson	15
3.1	Kết quả chất lượng của mô hình baseline	24
3.2	Kết quả các cấu hình pruning	25
3.3	Ma trận cấu hình tối ưu mô hình	27
3.4	Quyết định lựa chọn model triển khai	29
4.1	Yêu cầu chức năng của nền tảng	30
4.2	Bộ 8 đặc trưng telemetry chính thu thập từ Edge Node	38
5.1	Các thành phần phần mềm của môi trường triển khai	43
6.1	Cấu hình thiết bị thực nghiệm	51
6.2	Kết quả accuracy của các cấu hình tối ưu	53
6.3	YOLO inference trên Jetson Nano	55
6.4	YOLO inference trên Jetson Orin Nano	57
6.5	So sánh hiệu năng giữa Jetson Nano và Jetson Orin Nano	58
6.6	Benchmark pipeline Edge Runtime	58
6.7	Benchmark Edge Agent	59
6.8	Benchmark Model Deployment	59
6.9	Benchmark Monitoring và Incident	59
6.10	Benchmark Knowledge và Copilot	60
6.11	Kết quả kịch bản end-to-end	61
B.1	Các bảng hoặc collection nghiệp vụ chính	68
C.1	Nhóm API tham chiếu	69
D.1	Danh sách MCP tool tham chiếu	70

F.1	Cấu hình tối thiểu của một lần benchmark	72
-----	--	----

DANH SÁCH CÁC KÝ HIỆU, CHỮ VIẾT TẮT

Ký hiệu	Ý nghĩa
AI	Artificial Intelligence – Trí tuệ nhân tạo
API	Application Programming Interface – Giao diện lập trình ứng dụng
CUDA	Compute Unified Device Architecture – Nền tảng tính toán GPU của NVIDIA
CPU	Central Processing Unit – Bộ xử lý trung tâm
DDD	Domain-Driven Design – Thiết kế hướng miền
GPU	Graphics Processing Unit – Bộ xử lý đồ họa
NPU	Neural Processing Unit – Bộ xử lý mạng nơ-ron
Edge AI	Thực thi tác vụ trí tuệ nhân tạo trên hoặc gần nguồn sinh dữ liệu
FP16	Floating Point 16-bit – Số thực dấu phẩy động 16 bit
FP32	Floating Point 32-bit – Số thực dấu phẩy động 32 bit
FPS	Frames Per Second – Số khung hình xử lý trong một giây
INT8	Integer 8-bit – Số nguyên 8 bit
IoU	Intersection over Union – Tỷ lệ giao trên hợp
LLM	Large Language Model – Mô hình ngôn ngữ lớn
mAP	mean Average Precision – Độ chính xác trung bình trên các lớp
MCP	Model Context Protocol – Giao thức kết nối mô hình với công cụ và dữ liệu
MQTT	Message Queuing Telemetry Transport – Giao thức truyền thông điệp nhẹ
ONNX	Open Neural Network Exchange – Định dạng trao đổi mô hình học máy
PTQ	Post-Training Quantization – Lượng tử hóa sau huấn luyện
QAT	Quantization-Aware Training – Huấn luyện có nhận biết lượng tử hóa
RAG	Retrieval-Augmented Generation – Sinh nội dung có tăng cường truy xuất
RAM	Random Access Memory – Bộ nhớ truy cập ngẫu nhiên
RBAC	Role-Based Access Control – Kiểm soát truy cập theo vai trò

Ký hiệu	Ý nghĩa
REST	Representational State Transfer – Phong cách kiến trúc dịch vụ web
TLS	Transport Layer Security – Giao thức bảo mật tầng truyền tải
YOLO	You Only Look Once – Họ mô hình phát hiện đối tượng một giai đoạn

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

Chương này trình bày bối cảnh hình thành đề tài, những khó khăn khi đưa mô hình trí tuệ nhân tạo xuống thiết bị biên và nhu cầu quản lý tập trung một đội thiết bị phân tán. Từ đó, chương xác định mục tiêu, phạm vi, đối tượng, phương pháp nghiên cứu và các đóng góp dự kiến của đồ án.

1.1 Đặt vấn đề

1.1.1 Sự phát triển của AI tại Edge

Các hệ thống thị giác máy tính trước đây thường truyền dữ liệu từ camera về máy chủ hoặc trung tâm dữ liệu để xử lý. Cách tiếp cận này thuận tiện cho việc tập trung tài nguyên nhưng tạo ra độ trễ truyền thông, tiêu thụ băng thông và làm tăng rủi ro lộ dữ liệu. Edge AI đưa mô hình suy luận đến gần nguồn sinh dữ liệu, cho phép thiết bị phản hồi ngay cả khi kết nối mạng không ổn định và chỉ gửi về đám mây các kết quả đã được chọn lọc.

Sự phổ biến của GPU nhúng, NPU và các bộ thực thi tối ưu đã mở rộng phạm vi ứng dụng Edge AI trong giám sát an toàn, giao thông, nông nghiệp, bán lẻ và tự động hóa công nghiệp. Trong các ứng dụng này, mô hình phát hiện đối tượng thuộc họ YOLO phù hợp nhờ khả năng xử lý một giai đoạn và cân bằng tương đối tốt giữa độ chính xác với tốc độ [1, 3]. Tuy nhiên, một mô hình chạy được trên máy trạm chưa đồng nghĩa với việc có thể vận hành ổn định trên thiết bị biên.

1.1.2 Hạn chế của triển khai AI trên Edge Device

Thiết bị biên bị giới hạn đồng thời bởi nhiều loại tài nguyên. Các giới hạn này có quan hệ với nhau và phải được xem xét trên toàn bộ chuỗi suy luận thay vì chỉ đo thời gian thực thi của riêng mạng nơ-ron.

- **Tài nguyên GPU:** số nhân xử lý và thông lượng tính toán thấp hơn máy chủ, trong khi GPU còn có thể được chia sẻ cho giải mã video, hiển thị và các tác vụ khác.
- **Bộ nhớ:** dung lượng RAM và bộ nhớ GPU giới hạn kích thước mô hình, batch size, số luồng camera và lượng bộ đệm có thể duy trì.
- **Điện năng:** thiết bị thường hoạt động theo power mode cố định; tăng xung nhịp có thể cải thiện FPS nhưng làm tăng công suất tiêu thụ.

- **Nhiệt độ:** tải suy luận kéo dài có thể làm thiết bị giảm xung để bảo vệ phần cứng, khiến hiệu năng thực tế thấp hơn phép đo ngắn hạn.
- **Yêu cầu thời gian thực:** hệ thống phải kiểm soát cả độ trễ trung bình, độ trễ phần vi cao và độ trễ đầu cuối từ camera đến kết quả.

Vì vậy, bài toán triển khai cần tối ưu đồng thời độ chính xác, tốc độ, kích thước mô hình, bộ nhớ và điện năng. Mọi so sánh phải được thực hiện trên cùng dữ liệu, cùng kích thước đầu vào, cùng thiết bị và cùng quy trình đo.

1.1.3 Bài toán quản lý và triển khai AI Model trên nhiều Edge Device

Khi số lượng thiết bị tăng, việc sao chép thủ công tệp mô hình và khởi động tiến trình qua từng phiên SSH không còn phù hợp. Người vận hành cần biết thiết bị nào đang trực tuyến, phiên bản mô hình đang chạy, cấu hình mong muốn, trạng thái thực tế và lịch sử thay đổi. Một lỗi triển khai phải được phát hiện, khoanh vùng và khôi phục mà không làm gián đoạn toàn bộ đội thiết bị.

Bài toán vì thế bao gồm cả mặt phẳng dữ liệu tại Edge và mặt phẳng điều khiển trên Cloud. Cloud Control Plane quản lý danh tính thiết bị, Model Registry, Deployment, Command, Monitoring và Incident. Edge Agent tiếp nhận desired state, tải artifact, kiểm tra tính toàn vẹn, điều khiển AI Runtime và báo cáo telemetry. Mỗi thao tác phải có phiên bản, trạng thái và dấu vết kiểm toán để có thể truy nguyên khi xảy ra sự cố.

1.1.4 Sự cần thiết của tối ưu mô hình AI

Mua phần cứng mạnh hơn chỉ giải quyết một phần vấn đề và làm tăng chi phí khi quy mô đội thiết bị mở rộng. Tối ưu mô hình giúp khai thác hiệu quả tài nguyên hiện có, tăng số luồng camera có thể phục vụ và giảm thời gian phản hồi. Trong đề án này, pruning được dùng để loại bỏ phần tham số hoặc cấu trúc ít quan trọng; Quantization-Aware Training (QAT) giúp mô hình thích nghi với sai số lượng tử; TensorRT chuyển đổi đồ thị sang engine tối ưu cho GPU NVIDIA.

Tối ưu chỉ có ý nghĩa khi mức giảm chi phí suy luận không làm chất lượng nhận diện suy giảm vượt ngưỡng chấp nhận. Do đó, đề án không tìm mô hình nhỏ nhất hoặc nhanh nhất một cách độc lập mà tìm cấu hình có trade-off phù hợp giữa accuracy, latency, FPS, model size, GPU memory và power trên Jetson Nano và Jetson Orin Nano.

1.2 Mục tiêu đề tài

1.2.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề án là xây dựng nền tảng quản lý và vận hành AI tại Edge, bao phủ vòng đời từ tối ưu mô hình đến triển khai, giám sát, xử lý sự cố và hỗ trợ vận hành bằng AI Copilot. Luồng tổng thể được thể hiện trong Hình 1.1.



Hình 1.1: Vòng đời AI tại Edge được nghiên cứu trong đề án

Nền tảng phải tạo được mối liên kết giữa bằng chứng thực nghiệm của một phiên bản mô hình với artifact được phát hành và trạng thái chạy thực tế trên từng thiết bị. Nhờ đó, quyết định triển khai hoặc rollback không chỉ dựa trên tên tệp mà dựa trên phiên bản, cấu hình tương thích, chỉ số benchmark và telemetry vận hành.

1.2.2 Mục tiêu cụ thể

Các mục tiêu cụ thể của đề án gồm:

1. Xây dựng và đánh giá mô hình YOLO baseline trên tập dữ liệu mục tiêu.
2. Tối ưu YOLO bằng pruning với nhiều pruning ratio.
3. Tối ưu YOLO bằng Quantization-Aware Training.
4. Kết hợp pruning, fine-tuning và QAT trong một pipeline có thể lặp lại.
5. Chuyển đổi mô hình sang ONNX và TensorRT ở chế độ FP16, INT8.
6. Đánh giá chất lượng và hiệu năng trên Jetson Nano.
7. Đánh giá chất lượng và hiệu năng trên Jetson Orin Nano.
8. Xây dựng Edge Runtime thực hiện chuỗi camera–inference–result.
9. Xây dựng Cloud Control Plane và Edge Agent quản lý đội thiết bị.
10. Xây dựng cơ chế Model Registry, Deployment, Monitoring và Incident Management có khả năng rollback.
11. Xây dựng AI Copilot sử dụng Knowledge và công cụ có kiểm soát để hỗ trợ truy vấn, chẩn đoán và đề xuất thao tác vận hành.

Mỗi mục tiêu được gắn với đầu ra có thể kiểm tra, gồm mã nguồn, artifact mô hình, cấu hình triển khai, số liệu benchmark, lịch sử lệnh và kịch bản trình diễn xuyên suốt.

1.3 Phạm vi đề tài

Đề án tập trung vào bài toán phát hiện đối tượng bằng YOLO trên hai thiết bị đại diện là Jetson Nano và Jetson Orin Nano. Phạm vi tối ưu gồm pruning, fine-tuning, QAT, xuất ONNX và tạo TensorRT engine. Các phép đo tập trung vào accuracy, latency, FPS, model size, GPU memory, mức sử dụng tài nguyên và công suất theo khả năng thu thập của thiết bị thực tế.

Ở tầng nền tảng, đề án xây dựng các chức năng cốt lõi cho Fleet, Model Registry, Deployment, Command, Runtime, Monitoring, Incident, Knowledge và Copilot. Hệ thống được đánh giá trong quy mô phòng thí nghiệm với [CẦN BỔ SUNG: số lượng thiết bị thực tế], chưa hướng đến thay thế một nền tảng MLOps thương mại ở quy mô hàng nghìn node.

Đề án không đặt mục tiêu nghiên cứu một kiến trúc YOLO mới, thiết kế phần cứng tăng tốc mới hoặc xây dựng mô hình ngôn ngữ nền tảng. Federated learning, continuous learning, orchestration đa cụm và auto-remediation hoàn toàn tự động được xem là hướng phát triển sau khi nền tảng cốt lõi đã được kiểm chứng.

1.4 Đối tượng nghiên cứu

Đối tượng nghiên cứu thứ nhất là mô hình phát hiện đối tượng YOLO và ảnh hưởng của pruning, QAT, FP16, INT8 đến chất lượng nhận diện cũng như hiệu năng suy luận. Đối tượng thứ hai là chuỗi Edge Runtime, bao gồm thu nhận ảnh, tiền xử lý, thực thi TensorRT, hậu xử lý và xuất kết quả.

Đối tượng thứ ba là kiến trúc nền tảng phân tán Cloud–Edge: Control Plane, Edge Agent, giao thức lệnh, desired state, actual state, telemetry, vòng đời model artifact và incident. Đối tượng thứ tư là AI Copilot sử dụng RAG, tool calling và MCP để truy cập tri thức và dữ liệu vận hành trong phạm vi quyền được cấp.

1.5 Phương pháp nghiên cứu

Đề án kết hợp nghiên cứu tài liệu, thực nghiệm mô hình và thiết kế hệ thống. Các tài liệu về Edge AI, YOLO, pruning, quantization, TensorRT, hệ thống phân tán, RAG và Domain-Driven Design được tổng hợp để hình thành cơ sở lý thuyết và các giả thuyết cần kiểm chứng.

Đối với mô hình, phương pháp thực nghiệm có kiểm soát được sử dụng: giữ cố

định tập dữ liệu, cách chia tập, kích thước ảnh và tiêu chí đo; sau đó thay đổi một nhóm yếu tố như pruning ratio hoặc precision. Mỗi cấu hình được đo lặp lại, ghi warm-up, số vòng đo và điều kiện power mode để có thể tái lập.

Đối với nền tảng, đề án áp dụng Domain-Driven Design để phân chia miền nghiệp vụ, sau đó hiện thực nguyên mẫu theo kiến trúc Cloud Control Plane–Edge Agent. Các kịch bản kiểm thử bao gồm triển khai thành công, mất kết nối, artifact lỗi, runtime lỗi, rollback và phát sinh incident. AI Copilot được đánh giá bằng bộ câu hỏi và nhiệm vụ có đáp án mong đợi, đồng thời kiểm tra việc tuân thủ quyền và yêu cầu phê duyệt.

1.6 Đóng góp của đề tài

Các đóng góp chính dự kiến của đề án gồm:

1. Một quy trình thực nghiệm thống nhất để so sánh YOLO baseline, pruning, QAT, pruning kết hợp QAT và TensorRT trên hai thế hệ Jetson.
2. Một cơ chế đóng gói model artifact kèm phiên bản, hàm băm, cấu hình tiền xử lý–hậu xử lý, thông tin tương thích và kết quả benchmark.
3. Một nền tảng Cloud–Edge quản lý Fleet, Model Registry, Deployment, Runtime, Monitoring và Incident theo desired state và actual state.
4. Một Edge Agent hỗ trợ giao tiếp bất đồng bộ, cập nhật mô hình nguyên tử, giám sát runtime, bộ đệm telemetry và rollback khi triển khai thất bại.
5. Một AI Copilot kết hợp Knowledge Retrieval với công cụ nghiệp vụ, cho phép giải thích trạng thái và lập kế hoạch thao tác có kiểm soát.
6. Một bộ kịch bản đánh giá xuyên suốt từ tối ưu mô hình đến triển khai, phát hiện suy giảm FPS, tạo incident, phân tích và rollback.

1.7 Cấu trúc đề án

Ngoài phần đầu báo cáo, tài liệu tham khảo và phụ lục, đề án gồm bảy chương. Chương 1 trình bày bối cảnh, mục tiêu, phạm vi và đóng góp. Chương 2 hệ thống hóa cơ sở lý thuyết theo pipeline từ Edge AI, YOLO, nén và lượng tử mô hình đến TensorRT, triển khai phân tán, RAG, AI Agent và Domain-Driven Design.

Chương 3 trình bày nghiên cứu tối ưu YOLO, xây dựng baseline, pruning, QAT, kết hợp pruning–QAT và lựa chọn model triển khai. Chương 4 phân tích yêu cầu, thiết kế domain và kiến trúc nền tảng Edge AI. Chương 5 mô tả quá trình xây dựng Cloud, Edge Agent, Edge Runtime, Monitoring, giao diện và AI Copilot.

Chương 6 trình bày môi trường thực nghiệm, benchmark mô hình trên Jetson Nano và Jetson Orin Nano, benchmark toàn pipeline và đánh giá nền tảng. Chương 7 tổng kết kết quả, phân tích hạn chế và đề xuất hướng phát triển.

1.8 Kết chương

Chương 1 đã xác định đề án là bài toán kết hợp giữa nghiên cứu tối ưu YOLO và xây dựng nền tảng vận hành AI phân tán. Các mục tiêu không dừng ở việc tăng FPS mà bao phủ toàn bộ vòng đời model từ huấn luyện, đóng gói, triển khai đến giám sát và xử lý sự cố. Chương tiếp theo trình bày các kiến thức nền tảng cần thiết để thiết kế và đánh giá giải pháp.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương này tổ chức cơ sở lý thuyết theo đúng pipeline của đề tài: từ đặc trưng Edge AI, mô hình YOLO, các kỹ thuật nén và lượng tử hóa, TensorRT và Jetson đến quản lý model deployment, Edge Runtime, nền tảng phân tán, RAG, AI Agent và Domain-Driven Design. Cách sắp xếp này tạo cầu nối trực tiếp từ kiến thức nền tảng sang nghiên cứu tối ưu ở Chương 3 và thiết kế nền tảng ở Chương 4.

2.1 Edge AI

2.1.1 *Khái niệm Edge AI*

Edge AI là cách triển khai tác vụ trí tuệ nhân tạo trên thiết bị đặt gần nguồn sinh dữ liệu, thay vì gửi toàn bộ dữ liệu thô về đám mây. Thiết bị có thể là gateway, máy tính nhúng, camera thông minh hoặc máy công nghiệp. Lợi ích chính gồm giảm độ trễ, giảm băng thông, tăng khả năng hoạt động ngoại tuyến và hạn chế dữ liệu nhạy cảm rời khỏi hiện trường [10, 11].

Edge AI không loại bỏ vai trò của Cloud. Cloud vẫn phù hợp cho huấn luyện, lưu trữ artifact, quản lý đội thiết bị và phân tích dài hạn; Edge chịu trách nhiệm cho vòng lặp phản hồi nhanh. Do đó, giải pháp thực tế thường là một hệ thống lai trong đó Cloud và Edge có nhiệm vụ khác nhau nhưng chia sẻ cùng vòng đời model.

2.1.2 *Kiến trúc Cloud–Edge*

Kiến trúc Cloud–Edge gồm ba lớp chính: Cloud Control Plane, kênh truyền thông và Edge Plane. Control Plane lưu desired state, model version, chính sách và lịch sử vận hành. Edge Plane chứa Edge Agent và AI Runtime, duy trì actual state và tiếp tục suy luận khi mất kết nối. Kênh truyền thông mang lệnh, sự kiện và telemetry; các tệp model dung lượng lớn có thể được phân phối qua object storage.

Kiến trúc cần chấp nhận kết nối không ổn định. Lệnh phải có định danh duy nhất, thời hạn hiệu lực và khả năng thực thi lặp an toàn. Edge Agent phải lưu trạng thái cục bộ, đồng bộ lại khi có mạng và không được dừng runtime đang khỏe chỉ vì Control Plane tạm thời không truy cập được.

2.1.3 Đặc điểm của Edge Device

Edge Device có tài nguyên nhỏ hơn máy chủ, thường dùng bộ nhớ chia sẻ giữa CPU và GPU, giới hạn điện năng và tản nhiệt. Phần mềm phụ thuộc chặt vào phiên bản hệ điều hành, CUDA, cuDNN, TensorRT và driver. Cùng một model artifact có thể không chạy được trên hai thiết bị nếu engine được tạo cho kiến trúc GPU hoặc phiên bản runtime khác nhau.

Thiết bị biên cũng nằm ngoài trung tâm dữ liệu nên dễ gặp mất nguồn, mất mạng, camera lỗi hoặc thay đổi môi trường. Vì vậy, khả năng quan sát, tự khởi động lại, rollback và xác thực artifact là một phần của yêu cầu hệ thống chứ không phải chức năng bổ sung.

2.1.4 Các vấn đề khi triển khai AI trên Edge

Năm nhóm vấn đề chính là năng lực tính toán, bộ nhớ, latency, power và thermal. Chúng tạo thành bài toán tối ưu đa mục tiêu: giảm số phép tính nhưng không làm accuracy giảm quá mức; giảm precision nhưng vẫn duy trì tính ổn định số; tăng FPS nhưng không gây quá nhiệt hoặc vượt ngân sách điện.

Ngoài model inference, hệ thống còn phải tính chi phí camera capture, chuyển đổi màu, resize, chuẩn hóa, non-maximum suppression, truyền kết quả và telemetry. Do đó, benchmark chỉ đo thời gian TensorRT engine là cần thiết nhưng chưa đủ để kết luận về khả năng đáp ứng thời gian thực của toàn hệ thống.

2.2 Object Detection và YOLO

2.2.1 Bài toán Object Detection

Object Detection xác định đồng thời lớp và vị trí của các đối tượng trong ảnh. Đầu ra thường là tập các bounding box $b_i = (x_i, y_i, w_i, h_i)$, nhãn lớp và độ tin cậy. Khác với phân loại ảnh, một ảnh có thể chứa nhiều đối tượng và mô hình phải giải quyết cả bài toán định vị lẫn phân loại.

Độ chồng lấp giữa hộp dự đoán A và hộp chuẩn B được đo bằng Intersection over Union:

$$IoU(A, B) = \frac{|A \cap B|}{|A \cup B|}. \quad (2.1)$$

Một dự đoán được xem là đúng khi lớp phù hợp và IoU vượt ngưỡng đánh giá.

2.2.2 Kiến trúc YOLO

YOLO là họ mô hình phát hiện đối tượng một giai đoạn, thực hiện trích xuất đặc trưng và dự đoán trong một đồ thị thống nhất [1, 2]. Kiến trúc tổng quát gồm backbone để trích xuất đặc trưng, neck để tổng hợp đặc trưng đa tỉ lệ và detection head để dự đoán vị trí, objectness và lớp.

Các biến thể YOLO khác nhau ở block tích chập, cơ chế kết hợp đặc trưng, hàm loss và cách biểu diễn detection head. Khi tối ưu, cần bảo toàn những tầng ảnh hưởng mạnh đến đầu ra, đồng thời ưu tiên cắt tỉa cấu trúc mà TensorRT có thể khai thác thành giảm thời gian thực thi thực tế.

2.2.3 Quy trình huấn luyện YOLO

Quy trình huấn luyện gồm chuẩn hóa nhãn, chia train-validation-test, tăng cường dữ liệu, khởi tạo trọng số, tối ưu loss và chọn checkpoint. Mọi thí nghiệm phải lưu seed, phiên bản mã nguồn, cấu hình optimizer, learning rate, batch size, image size và số epoch để có thể tái lập.

Tập test chỉ được dùng cho đánh giá cuối. Các quyết định về pruning ratio, lịch fine-tuning và ngưỡng dừng sớm phải dựa trên tập validation nhằm tránh điều chỉnh gián tiếp theo tập test.

2.2.4 Các chỉ số đánh giá

Với TP , FP và FN lần lượt là số dự đoán đúng, dương tính giả và âm tính giả, Precision và Recall được xác định bởi:

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}. \quad (2.2)$$

Average Precision là diện tích dưới đường Precision-Recall của một lớp. mAP là trung bình AP trên các lớp. $mAP@50$ dùng ngưỡng IoU 0,5; $mAP@50:95$ lấy trung bình trên các ngưỡng từ 0,5 đến 0,95 với bước 0,05. Đồ án báo cáo đồng thời Precision, Recall, $mAP@50$ và $mAP@50:95$ để tránh kết luận dựa trên một chỉ số đơn lẻ.

2.3 Model Compression

2.3.1 Tổng quan Model Compression

Model Compression là nhóm kỹ thuật giảm chi phí lưu trữ hoặc suy luận của mạng nơ-ron. Các hướng phổ biến gồm pruning, quantization, knowledge distillation và thiết kế kiến trúc gọn. Đề án tập trung vào pruning và quantization vì hai kỹ thuật này có thể áp dụng trực tiếp lên baseline và kết hợp với TensorRT.

Hiệu quả nén phải được đánh giá trên cả kích thước artifact, số phép tính lý thuyết và latency thực tế. Một mô hình có nhiều trọng số bằng không chưa chắc chạy nhanh hơn nếu backend vẫn thực hiện phép tính dense.

2.3.2 Model Pruning

Pruning loại bỏ trọng số, kênh hoặc cấu trúc được xem là ít quan trọng. Tiêu chí có thể dựa trên độ lớn trọng số, chuẩn của filter, độ nhảy loss hoặc thông tin gradient. Quy trình cơ bản gồm huấn luyện baseline, tính điểm quan trọng, cắt tỉa, fine-tuning và đánh giá lại [5, 6].

2.3.3 Structured Pruning

Structured pruning loại bỏ đơn vị có cấu trúc như channel, filter hoặc block. Kết quả là tensor có kích thước nhỏ hơn và có thể được thư viện dense hiện có khai thác mà không cần kernel thưa chuyên dụng. Đây là lựa chọn phù hợp khi mục tiêu cuối là TensorRT trên Jetson.

Nhược điểm là mỗi quyết định loại bỏ channel ảnh hưởng đến các tầng lân cận và có thể làm accuracy giảm nhanh. Việc cắt tỉa cần tôn trọng quan hệ residual, concatenation và detection head trong kiến trúc YOLO.

2.3.4 Unstructured Pruning

Unstructured pruning đặt các trọng số riêng lẻ về không và có thể đạt sparsity cao với mức suy giảm accuracy thấp hơn. Tuy nhiên, kích thước tensor không đổi; latency chỉ giảm rõ khi phần cứng và runtime hỗ trợ sparse kernel tương ứng.

Trong đề án, unstructured pruning có thể được dùng làm đối chứng về sparsity, nhưng cấu hình triển khai ưu tiên structured pruning nếu mục tiêu là giảm latency thực

tế trên Jetson.

2.3.5 *Pruning Ratio*

Pruning ratio biểu diễn tỷ lệ phần tử hoặc cấu trúc bị loại bỏ:

$$r = \frac{N_{before} - N_{after}}{N_{before}} \times 100\%. \quad (2.3)$$

Các mức 20%, 30% và 40% được dùng làm điểm thực nghiệm ban đầu. Tỷ lệ này không mặc định áp dụng đồng đều cho mọi tầng; các tầng đầu, tầng đầu ra và nhánh nhảy cảm có thể được bảo vệ hoặc đặt ngưỡng riêng.

2.3.6 *Fine-tuning sau Pruning*

Sau pruning, phân bố đặc trưng bị thay đổi nên mô hình cần được fine-tune với learning rate nhỏ hơn baseline. Fine-tuning phục hồi accuracy trong khi duy trì cấu trúc đã cắt. Checkpoint tốt nhất được chọn theo validation mAP và phải được đánh giá lại trên tập test trước khi chuyển sang QAT hoặc TensorRT.

2.4 Model Quantization

2.4.1 *Khái niệm Quantization*

Quantization ánh xạ giá trị số thực có độ chính xác cao sang biểu diễn có số bit thấp hơn. Với lượng tử tuyến tính, một giá trị x được ánh xạ thành số nguyên q thông qua scale s và zero-point z :

$$q = \text{clip} \left(\text{round} \left(\frac{x}{s} \right) + z \right). \quad (2.4)$$

Việc giảm số bit làm giảm dung lượng và có thể tăng thông lượng, nhưng gây sai số lượng tử cần được kiểm soát.

2.4.2 *FP32, FP16 và INT8*

FP32 là biểu diễn phổ biến khi huấn luyện và được dùng làm chuẩn chất lượng. FP16 giảm một nửa dung lượng mỗi phần tử, phù hợp với Tensor Core và thường giữ

accuracy gần FP32. INT8 dùng số nguyên 8 bit, có tiềm năng giảm bộ nhớ và tăng tốc mạnh hơn nhưng nhạy với dải giá trị activation và yêu cầu calibration hoặc QAT.

2.4.3 *Post-Training Quantization*

Post-Training Quantization thực hiện lượng tử sau khi mô hình đã huấn luyện. PTQ không yêu cầu huấn luyện lại hoặc chỉ cần calibration dataset để ước lượng dải activation. Ưu điểm là quy trình nhanh; hạn chế là accuracy có thể giảm với tầng nhạy cảm, dữ liệu lệch hoặc mô hình nhỏ.

2.4.4 *Quantization-Aware Training*

QAT mô phỏng hiệu ứng lượng tử trong quá trình huấn luyện để trọng số thích nghi với sai số làm tròn và clipping [4]. Forward pass sử dụng các nút fake quantization, còn gradient được xấp xỉ để cập nhật trọng số thực. QAT tốn thêm thời gian huấn luyện nhưng thường duy trì accuracy INT8 tốt hơn PTQ.

2.4.5 *Fake Quantization*

Fake quantization lượng tử và giải lượng tử tensor trong forward pass nhưng vẫn lưu trọng số ở dạng số thực để tối ưu. Các observer thu thập thống kê min-max hoặc histogram nhằm xác định scale. Khi export, các thông tin này được chuyển thành toán tử lượng tử hoặc metadata phục vụ TensorRT.

2.4.6 *So sánh PTQ và QAT*

Bảng 2.1: So sánh PTQ và QAT

Tiêu chí	PTQ	QAT
Dữ liệu cần thiết	Tập calibration đại diện	Dữ liệu huấn luyện/validation
Chi phí thực hiện	Thấp, không huấn luyện lại	Cao hơn do fine-tuning có fake quant
Khả năng giữ accuracy	Phụ thuộc độ nhạy mô hình	Thường tốt hơn với INT8
Độ phức tạp pipeline	Đơn giản	Cần cấu hình và export tương thích
Vai trò trong đồ án	Baseline INT8 tham chiếu	Phương pháp tối ưu chính

2.5 TensorRT

2.5.1 Kiến trúc TensorRT

TensorRT là bộ SDK tối ưu và thực thi suy luận cho GPU NVIDIA. Builder tiếp nhận đồ thị mạng, áp dụng tối ưu như fusion, lựa chọn kernel và precision, sau đó tạo serialized engine. Runtime nạp engine, cấp phát tensor và thực thi qua execution context [9].

Engine phụ thuộc vào kiến trúc GPU, phiên bản TensorRT, plugin và shape profile. Vì vậy, manifest của model artifact phải ghi rõ môi trường tạo engine và điều kiện tương thích.

2.5.2 ONNX

ONNX là định dạng trung gian giúp trao đổi mô hình giữa framework huấn luyện và runtime suy luận [8]. Pipeline của đồ án export checkpoint sang ONNX, kiểm tra graph, so sánh đầu ra với framework gốc rồi mới xây dựng TensorRT engine. Các toán tử không được hỗ trợ phải được thay thế, phân rã hoặc hiện thực bằng plugin.

2.5.3 TensorRT Engine

TensorRT engine là artifact đã được tối ưu cho một tập cấu hình cụ thể. Quá trình build có thể lựa chọn tactic khác nhau theo workspace và phần cứng. Engine cần đi kèm input shape, data type, tên binding, preprocessing, postprocessing và hàm băm để Edge Runtime tải đúng cách.

2.5.4 FP16 Inference

FP16 inference cho phép TensorRT dùng kernel bán chính xác ở các tầng được hỗ trợ và quay về precision khác khi cần. Đây thường là cấu hình triển khai đầu tiên vì quy trình chuyển đổi đơn giản hơn INT8 và mức suy giảm accuracy nhỏ. Kết quả vẫn phải được so sánh với FP32 trên cùng tập test.

2.5.5 INT8 Inference

INT8 inference yêu cầu scale activation phù hợp. Với PTQ, TensorRT dùng calibrator; với QAT, thông tin lượng tử được mang theo từ graph nếu pipeline export hỗ

trợ. Một số tầng có thể giữ FP16 hoặc FP32 để bảo toàn accuracy, tạo ra engine mixed precision.

2.5.6 *TensorRT trên NVIDIA Jetson*

Trên Jetson, TensorRT được phân phối cùng hệ sinh thái JetPack và sử dụng CUDA, cuDNN cùng driver tương ứng. Việc build engine trực tiếp trên thiết bị đích giúp giảm rủi ro không tương thích. Đồ án lưu riêng engine cho Jetson Nano và Jetson Orin Nano, thay vì giả định một engine có thể dùng chung cho mọi GPU.

2.6 NVIDIA Jetson

2.6.1 *Jetson Nano*

Jetson Nano đại diện cho nhóm thiết bị Edge AI có tài nguyên hạn chế. Thiết bị phù hợp để đánh giá mức tối thiểu mà model có thể vận hành, đồng thời làm rõ ảnh hưởng của bộ nhớ, power mode và thermal throttling. Phiên bản JetPack, CUDA, TensorRT, dung lượng RAM và chế độ công suất phải được ghi theo đúng thiết bị thực nghiệm, vì các biến thể phần cứng có thể khác nhau.

2.6.2 *Jetson Orin Nano*

Jetson Orin Nano đại diện cho thế hệ GPU nhúng mới hơn, có kiến trúc và năng lực tăng tốc khác Jetson Nano. Thiết bị được dùng để đánh giá khả năng mở rộng hiệu năng của cùng pipeline model, đồng thời kiểm tra tính di động của Model Registry, Deployment và Edge Runtime giữa hai lớp node.

2.6.3 So sánh tài nguyên phần cứng

Bảng 2.2: Thông tin phần cứng cần ghi nhận cho hai thiết bị Jetson

Thành phần	Jetson Nano	Jetson Orin Nano
Kiến trúc GPU	[CẦN BỔ SUNG: thông tin thiết bị]	[CẦN BỔ SUNG: thông tin thiết bị]
Số nhân GPU/Tensor Core	[CẦN BỔ SUNG: thông tin thiết bị]	[CẦN BỔ SUNG: thông tin thiết bị]
RAM	[CẦN BỔ SUNG: dung lượng]	[CẦN BỔ SUNG: dung lượng]
JetPack	[CẦN BỔ SUNG: phiên bản]	[CẦN BỔ SUNG: phiên bản]
CUDA	[CẦN BỔ SUNG: phiên bản]	[CẦN BỔ SUNG: phiên bản]
TensorRT	[CẦN BỔ SUNG: phiên bản]	[CẦN BỔ SUNG: phiên bản]
Power mode	[CẦN BỔ SUNG: chế độ đo]	[CẦN BỔ SUNG: chế độ đo]
Tản nhiệt	[CẦN BỔ SUNG: cấu hình]	[CẦN BỔ SUNG: cấu hình]

Bảng 2.2 phải được điền từ lệnh hệ thống và nhãn thiết bị thực tế, không suy ra từ thông số quảng cáo của một phiên bản khác.

2.6.4 CUDA và GPU Acceleration

CUDA cung cấp mô hình lập trình và runtime để thực thi kernel trên GPU NVIDIA. Trong pipeline, TensorRT sử dụng CUDA để lập lịch kernel và truyền dữ liệu. Tối ưu end-to-end cần hạn chế sao chép CPU–GPU, tái sử dụng bộ đệm, dùng stream phù hợp và đồng bộ chỉ tại những điểm cần thiết.

GPU acceleration không tự động bảo đảm latency thấp. Kích thước input, batch, memory allocation, camera decode và hậu xử lý trên CPU có thể trở thành nút thắt. Vì vậy, đồ án đo từng giai đoạn của pipeline và tổng thời gian đầu cuối.

2.7 AI Model Deployment

2.7.1 Model Registry

Model Registry là kho quản lý metadata và artifact của các phiên bản mô hình. Mỗi bản ghi liên kết model name, version, framework, precision, input shape, chỉ số đánh giá, vị trí artifact, checksum và ma trận tương thích thiết bị. Registry không chỉ là thư mục lưu tệp mà là nguồn sự thật cho quyết định triển khai.

2.7.2 *Model Versioning*

Model versioning giúp phân biệt các checkpoint, cấu hình tối ưu và engine. Một phiên bản đã phát hành phải bất biến; thay đổi nội dung yêu cầu tạo phiên bản mới. Ngoài version dễ đọc, hệ thống lưu checksum để nhận diện chính xác bytes của artifact và ngăn trường hợp hai tệp khác nhau dùng chung tên.

2.7.3 *Model Artifact*

Artifact triển khai có thể gồm TensorRT engine, ONNX dự phòng, label map, manifest, cấu hình preprocessing–postprocessing và chữ ký hoặc checksum. Gói phải tự mô tả đủ để Edge Agent kiểm tra tính tương thích trước khi dùng runtime hiện tại.

2.7.4 *Deployment Strategy*

Chiến lược triển khai xác định thứ tự và phạm vi node nhận model. Các lựa chọn gồm triển khai đồng loạt, rolling deployment và canary. Với đội thiết bị phân tán, canary giúp kiểm tra một nhóm nhỏ trước khi mở rộng, đồng thời đặt ngưỡng dừng dựa trên health check, FPS, error rate hoặc nhiệt độ.

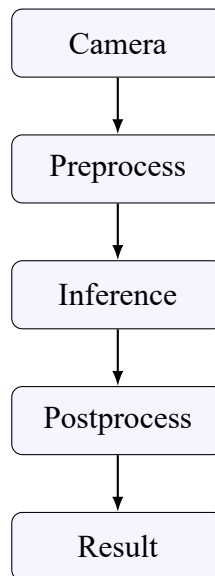
2.7.5 *Rollback*

Rollback đưa node về phiên bản đã biết là ổn định khi download, validation, activation hoặc health check thất bại. Edge Agent cần giữ ít nhất một artifact trước đó, cập nhật con trỏ active theo cách nguyên tử và báo cáo nguyên nhân. Cơ chế rollback phải được kiểm thử như một luồng bình thường, không chỉ dùng khi sự cố đã xảy ra.

2.8 Edge Runtime

2.8.1 *AI Inference Pipeline*

Edge Runtime biến frame từ camera thành detection theo chuỗi trong Hình 2.1.



Hình 2.1: Pipeline suy luận tại Edge

Preprocess phải khớp với huấn luyện về resize, letterbox, thứ tự kênh và chuẩn hóa. Postprocess giải mã tensor, áp dụng confidence threshold và non-maximum suppression. Sai khác ở hai bước này có thể làm giảm accuracy dù engine cho đầu ra số học chính xác.

2.8.2 Runtime Lifecycle

Runtime lifecycle gồm các trạng thái created, loading, ready, running, degraded, stopping, stopped và failed. Mọi chuyển trạng thái phải có điều kiện hợp lệ và reason code. Edge Agent không coi tiến trình đang tồn tại là đủ; health check cần xác nhận camera, engine và vòng xử lý frame đều hoạt động.

2.8.3 GPU Resource Management

Quản lý GPU bao gồm cấp phát bộ nhớ, số execution context, stream, batch size và số runtime đồng thời. Hệ thống cần từ chối hoặc trì hoãn cấu hình vượt khả năng node thay vì để thiết bị rơi vào out-of-memory. Model manifest và node capability được dùng để kiểm tra trước khi activation.

2.8.4 Runtime Monitoring

Runtime phát sinh các chỉ số FPS, latency từng stage, queue depth, frame drop, GPU usage, GPU memory, nhiệt độ, power và error count. Metrics cần gắn node, runtime,

model version và timestamp. Cửa sổ tổng hợp giúp phát hiện suy giảm kéo dài thay vì tạo incident từ một mẫu nhiễu đơn lẻ.

2.9 Distributed Edge Platform

2.9.1 Cloud Control Plane

Cloud Control Plane duy trì trạng thái mong muốn và điều phối các domain nghiệp vụ. Thành phần này cung cấp API cho Dashboard và Copilot, lưu lịch sử model, deployment, command, incident và audit. Control Plane không tham gia trực tiếp vào vòng lặp inference nên Edge vẫn hoạt động khi Cloud tạm thời gián đoạn.

2.9.2 Edge Agent

Edge Agent là đại diện quản lý trên mỗi node. Agent đăng ký danh tính, nhận lệnh, đồng bộ desired state, tải artifact, điều khiển runtime và gửi actual state. Agent phải giới hạn quyền hệ thống, xác thực nguồn lệnh và bảo đảm một command không bị thực thi hai lần khi message broker gửi lại.

2.9.3 Telemetry

Telemetry gồm metric, log, event và state report. Metric phục vụ theo dõi xu hướng; event ghi nhận thay đổi rời rạc; log hỗ trợ chẩn đoán chi tiết; state report đối chiếu desired state với actual state. Khi mất mạng, dữ liệu quan trọng được đệm cục bộ theo dung lượng giới hạn và gửi lại theo thứ tự phù hợp.

2.9.4 Message Broker

Message broker tách Control Plane khỏi kết nối trực tiếp đến từng node. MQTT phù hợp với thiết bị biên nhờ mô hình publish–subscribe và mức QoS linh hoạt [12]. Topic phải được phân vùng theo tenant/node, áp dụng TLS và quyền publish/subscribe tối thiểu.

2.9.5 Monitoring

Monitoring thu thập, lưu trữ và trực quan hóa telemetry; đồng thời đánh giá rule để phát hiện bất thường vận hành. Một rule cần metric, cửa sổ thời gian, ngưỡng, mức

ng nghiêm trọng và điều kiện đóng. Kết quả phát hiện được chuyển thành Incident thay vì chỉ tồn tại dưới dạng biểu đồ.

2.10 RAG và AI Agent

2.10.1 *Retrieval-Augmented Generation*

Retrieval-Augmented Generation (RAG) kết hợp truy xuất tài liệu với mô hình ngôn ngữ. Câu hỏi được dùng để tìm các đoạn liên quan; nội dung truy xuất được đưa vào ngữ cảnh để tạo câu trả lời có căn cứ [14]. Trong đồ án, Knowledge chứa runbook, hướng dẫn triển khai, mô tả lỗi và tài liệu hệ thống.

2.10.2 *Embedding Search*

Embedding search ánh xạ truy vấn và tài liệu thành vector rồi xếp hạng theo độ tương đồng. Phương pháp này tìm được nội dung gần nghĩa dù từ khóa không trùng khớp. Chất lượng phụ thuộc cách chia đoạn, mô hình embedding, metadata filter và ngưỡng lấy kết quả.

2.10.3 *Hybrid Search*

Hybrid search kết hợp tìm kiếm từ khóa với vector search. Từ khóa phù hợp với mã lỗi, model version và tên node; vector search phù hợp với câu hỏi diễn đạt tự nhiên. Hai danh sách có thể được chuẩn hóa điểm, hợp nhất và rerank trước khi đưa cho Agent.

2.10.4 *AI Agent*

AI Agent mở rộng RAG bằng khả năng lập kế hoạch và gọi công cụ. Agent có thể truy vấn Fleet, Runtime, Deployment hoặc Incident thay vì chỉ trả lời từ tài liệu. Mô hình ngôn ngữ không được truy cập trực tiếp cơ sở dữ liệu hay shell; mọi hành động đi qua tool schema, authorization và audit.

2.10.5 *Tool Calling*

Tool calling là cơ chế mô hình chọn hàm và tạo tham số có cấu trúc. Hệ thống phải kiểm tra schema, quyền người dùng, phạm vi node và policy trước khi thực thi. Các tool

thay đổi trạng thái dùng hai pha: tạo kế hoạch để người dùng xem xét, sau đó phê duyệt bằng định danh kế hoạch và token chống thực thi lại.

2.10.6 MCP

Model Context Protocol (MCP) chuẩn hóa cách AI application khám phá và gọi tools/resources do server cung cấp. Trong phạm vi đồ án, MCP được dùng làm biên tích hợp cho Knowledge và các công cụ vận hành. MCP không thay thế cơ chế xác thực hoặc phân quyền của domain; server vẫn phải xác minh danh tính và chính sách cho từng lần gọi.

2.11 Domain-Driven Design

Domain-Driven Design (DDD) tổ chức phần mềm quanh ngôn ngữ và quy tắc nghiệp vụ thay vì quanh bảng dữ liệu hoặc framework [15]. Cách tiếp cận này phù hợp với nền tảng có nhiều miền như Fleet, Model Registry, Deployment, Incident, Knowledge, Copilot và Ops.

2.11.1 Entity

Entity có định danh ổn định và vòng đời thay đổi theo thời gian. Ví dụ gồm Node, Model, Deployment, Command và Incident. Hai entity có cùng thuộc tính nhưng khác định danh vẫn là hai đối tượng khác nhau.

2.11.2 Value Object

Value Object được xác định bởi giá trị và thường bất biến, chẳng hạn ModelVersion, Checksum, DeviceCapability, MetricWindow hoặc DeploymentPolicy. Value Object giúp đóng gói validation và tránh truyền các chuỗi rời rạc không có ngữ nghĩa.

2.11.3 Aggregate

Aggregate là biên nhất quán cho một nhóm entity và value object. Aggregate root kiểm soát mọi thay đổi bên trong. Deployment có thể là aggregate quản lý target, batch, trạng thái và policy; các thay đổi phải đi qua hành vi của Deployment thay vì cập nhật bảng trực tiếp.

2.11.4 Repository

Repository cung cấp giao diện tải và lưu aggregate theo ngôn ngữ domain. Domain không phụ thuộc chi tiết PostgreSQL, object storage hoặc message broker. Điều này giúp kiểm thử quy tắc nghiệp vụ độc lập với hạ tầng.

2.11.5 Domain Service

Domain Service chứa hành vi nghiệp vụ không thuộc tự nhiên về một entity, ví dụ chọn nhóm canary dựa trên capability hoặc đánh giá chính sách triển khai. Service không nên trở thành nơi chứa toàn bộ logic; invariant vẫn được giữ trong aggregate khi có thể.

2.11.6 Bounded Context

Bounded Context xác định biên mô hình và ngôn ngữ. Fleet quản lý node và capability; Model Registry quản lý model/version/artifact; Deployment quản lý rollout; Incident quản lý sự cố; Knowledge quản lý tài liệu; Copilot điều phối hội thoại và tool call; Ops cung cấp quan sát vận hành. Các context trao đổi qua API hoặc event với hợp đồng rõ ràng.

2.12 Kết chương

Chương 2 đã trình bày chuỗi kiến thức từ model đến platform. YOLO, pruning, QAT và TensorRT cung cấp nền tảng cho nghiên cứu tối ưu; Model Registry, Deployment, Edge Runtime và nền tảng phân tán giải thích cách đưa model vào vận hành; RAG, AI Agent, MCP và DDD cung cấp cơ sở cho Knowledge, Copilot và cách chia domain. Chương 3 sử dụng các cơ sở này để xây dựng và đánh giá pipeline tối ưu YOLO.

CHƯƠNG 3. NGHIÊN CỨU VÀ TỐI ƯU MÔ HÌNH YOLO

Chương này trình bày phần nghiên cứu mô hình trước khi đưa model vào nền tảng. Quy trình bắt đầu từ baseline FP32, lần lượt đánh giá pruning, QAT, kết hợp pruning–QAT và chuyển đổi TensorRT. Kết quả của chương là một model version có trade-off phù hợp và đủ metadata để đăng ký vào Model Registry.

3.1 Bài toán tối ưu

3.1.1 Mô hình baseline

Mô hình baseline là checkpoint YOLO được huấn luyện theo cấu hình chuẩn trên tập dữ liệu mục tiêu và suy luận ở FP32. Baseline cung cấp mốc chất lượng, kích thước và hiệu năng để đánh giá mọi biến thể sau đó. Tên kiến trúc, phiên bản framework, trọng số khởi tạo và cấu hình huấn luyện phải được cố định trong manifest thí nghiệm.

3.1.2 Hạn chế của mô hình baseline

Baseline ưu tiên khả năng huấn luyện và chất lượng, chưa được tối ưu riêng cho Jetson. Các hạn chế có thể gồm kích thước trọng số lớn, nhiều phép tính, latency cao, GPU memory lớn và FPS chưa đạt yêu cầu. Mức độ hạn chế được xác nhận bằng số đo trên cả Jetson Nano và Jetson Orin Nano thay vì suy luận chỉ từ FLOPs.

3.1.3 Mục tiêu tối ưu

Đề án tối ưu đồng thời sáu tiêu chí: Accuracy, Latency, FPS, Model Size, GPU Memory và Power. Bài toán được biểu diễn dưới dạng:

$$\min_{m \in \mathcal{M}} (-A(m), L(m), -F(m), S(m), G(m), P(m)), \quad (3.1)$$

trong đó m là một cấu hình model, A là accuracy, L là latency, F là FPS, S là kích thước, G là GPU memory và P là công suất. Thay vì gộp tùy ý thành một điểm duy nhất, đề án ưu tiên phân tích tập nghiệm Pareto và các ràng buộc tối thiểu của ứng dụng.

Một cấu hình chỉ được xem là ứng viên triển khai khi mức giảm $mAP@50:95$ không

vượt [CẦN BỔ SUNG: ngưỡng chấp nhận], latency đầu cuối không vượt [CẦN BỔ SUNG: ngưỡng thời gian thực] và thiết bị hoạt động ổn định trong bài đo kéo dài.

3.2 Xây dựng mô hình YOLO baseline

3.2.1 Dataset

Tập dữ liệu sử dụng là [CẦN BỔ SUNG: tên và nguồn dataset], gồm [CẦN BỔ SUNG: số ảnh] ảnh và [CẦN BỔ SUNG: số lớp] lớp đối tượng. Dữ liệu được chia thành train, validation và test theo tỷ lệ [CẦN BỔ SUNG: tỷ lệ chia]. Nếu nhiều frame đến từ cùng video hoặc cùng hiện trường, các frame liên quan được giữ trong cùng một tập để tránh rò rỉ ngữ cảnh.

Trước huấn luyện, nhãn được kiểm tra về class id, bounding box vượt biên, ảnh lỗi, đối tượng quá nhỏ và mất cân bằng lớp. Báo cáo cần bổ sung thống kê số instance theo lớp, phân bố kích thước hộp và ví dụ dữ liệu đại diện.

3.2.2 Tiền xử lý dữ liệu

Ảnh được chuyển về kích thước [CẦN BỔ SUNG: image size], áp dụng letterbox và chuẩn hóa theo cấu hình của implementation YOLO. Các phép augmentation gồm [CẦN BỔ SUNG: liệt kê augmentation thực tế]. Cấu hình tiền xử lý được lưu để dùng lại khi export và trong Edge Runtime; mọi khác biệt về thứ tự kênh hoặc scale có thể làm kết quả suy luận sai lệch.

3.2.3 Huấn luyện

Baseline được huấn luyện với optimizer [CẦN BỔ SUNG: optimizer], learning rate [CẦN BỔ SUNG: giá trị], batch size [CẦN BỔ SUNG: giá trị] trong [CẦN BỔ SUNG: số epoch] epoch. Seed, phiên bản CUDA, framework và commit mã nguồn được lưu cùng checkpoint. Checkpoint tốt nhất được chọn theo validation $mAP@50:95$, không chọn theo tập test.

3.2.4 Đánh giá baseline

Bảng 3.1: Kết quả chất lượng của mô hình baseline

Model	Precision	Recall	mAP@50	mAP@50:95	Size (MB)
Baseline FP32	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]

Ngoài chỉ số tổng hợp, cần phân tích AP theo lớp và confusion matrix để nhận diện những lớp yếu. Kết quả này là đường chuẩn để xác định phần accuracy bị mất do pruning, quantization hoặc chuyển đổi runtime.

3.3 Tối ưu bằng Pruning

3.3.1 Lựa chọn phương pháp pruning

Đồ án ưu tiên structured channel pruning vì TensorRT có thể khai thác trực tiếp kích thước tensor đã giảm. Điểm quan trọng của channel được tính bằng [CẦN BỔ SUNG: chuẩn trọng số/gradient/tiêu chí thực tế]. Các tầng đầu vào, detection head và những nhánh có ràng buộc shape được bảo vệ hoặc cắt theo nhóm tương thích.

Để đánh giá lựa chọn này, một cấu hình unstructured pruning có thể được dùng làm đối chứng. Nếu sparsity tăng nhưng latency không giảm, kết quả cho thấy sự khác biệt giữa nén số tham số và tăng tốc thực tế.

3.3.2 Xác định pruning ratio

Ba mức khởi đầu là 20%, 30% và 40%. Với mỗi mức, pipeline tạo một cấu hình độc lập, ghi số channel còn lại, số tham số, FLOPs và checkpoint trước fine-tuning. Nếu mô hình mất ổn định ở một mức, quá trình dừng sớm và ghi rõ tầng gây lỗi thay vì tiếp tục tạo kết quả không hợp lệ.

3.3.3 Fine-tuning

Mô hình sau pruning được fine-tune từ [CẦN BỔ SUNG: số epoch] epoch với learning rate [CẦN BỔ SUNG: giá trị]. Mask hoặc cấu trúc đã cắt được khóa để phần tử bị loại bỏ không xuất hiện trở lại. Validation mAP được theo dõi theo epoch; checkpoint tốt nhất được dùng cho đánh giá test và làm đầu vào cho QAT.

3.3.4 Đánh giá mô hình sau pruning

Bảng 3.2: Kết quả các cấu hình pruning

Cấu hình	Ratio	Params	mAP@50: 95	Size	Nhận xét
Baseline	0%	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	Đường chuẩn
Pruned-20	20%	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Pruned-30	30%	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Pruned-40	40%	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]

Đánh giá pruning không kết luận chỉ từ bảng trên mà tiếp tục đo TensorRT latency. Cấu hình có ít tham số hơn nhưng tạo shape không thuận lợi cho kernel có thể không đem lại speedup tương ứng.

3.4 Tối ưu bằng QAT

3.4.1 Thiết lập quantization

QAT được cấu hình cho weight và activation theo scheme [CẦN BỔ SUNG: per-tensor hoặc per-channel, symmetric hoặc asymmetric]. Những toán tử chưa hỗ trợ INT8 hoặc nhảy với lượng tử được giữ ở precision cao hơn. Cấu hình quantization phải tương thích với đường export ONNX/TensorRT đã lựa chọn.

3.4.2 Fake Quantization

Các fake-quant node được chèn vào graph để mô phỏng clipping và làm tròn. Giai đoạn đầu thu thập thống kê activation; sau đó observer được đóng băng để scale ổn định trong phần cuối huấn luyện. Báo cáo lưu histogram hoặc min–max của các tầng nhảy cảm khi cần giải thích suy giảm accuracy.

3.4.3 QAT Training

QAT khởi tạo từ baseline hoặc checkpoint đã pruning và fine-tuning. Learning rate được đặt nhỏ hơn huấn luyện ban đầu; BatchNorm và observer được đóng băng theo lịch

[CẦN BỔ SUNG: lịch thực tế]. Quá trình theo dõi đồng thời loss, validation mAP và dấu hiệu mất ổn định số.

3.4.4 Export INT8

Checkpoint QAT được export sang ONNX với thông tin lượng tử tương ứng. Graph được kiểm tra bằng ONNX checker và chạy so sánh đầu ra trên một tập mẫu. Sau đó, TensorRT build engine INT8 hoặc mixed precision; log build và danh sách layer precision được lưu cùng artifact.

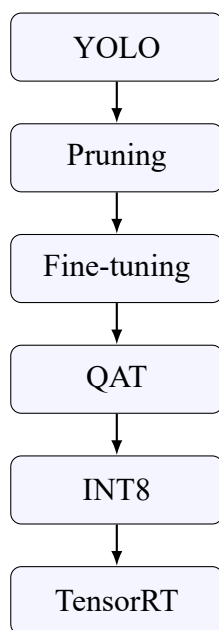
3.4.5 Đánh giá mô hình QAT

Mô hình QAT được đánh giá ở ba điểm: checkpoint fake-quant trong framework, ONNX sau export và TensorRT engine trên thiết bị. Chênh lệch giữa các điểm giúp xác định sai số đến từ lượng tử, export hay backend. Các chỉ số cần điền gồm Precision, Recall, $mAP@50$, $mAP@50:95$, latency, FPS và kích thước engine.

3.5 Kết hợp Pruning và QAT

3.5.1 Quy trình kết hợp

Quy trình kết hợp được thể hiện trong Hình 3.1. Pruning được thực hiện trước để thay đổi cấu trúc; fine-tuning phục hồi accuracy; QAT sau đó giúp mô hình thích nghi với INT8 trước khi tạo TensorRT engine.



Hình 3.1: Pipeline kết hợp pruning và QAT

3.5.2 Các cấu hình thực nghiệm

Bảng 3.3: Ma trận cấu hình tối ưu mô hình

Cấu hình	Pruning ratio	QAT	Precision đích
Baseline	0%	Không	FP32
Pruning 20%	20%	Không	FP32/FP16
Pruning 30%	30%	Không	FP32/FP16
Pruning 40%	40%	Không	FP32/FP16
QAT	0%	Có	INT8
Pruning 20% + QAT	20%	Có	INT8
Pruning 30% + QAT	30%	Có	INT8
Pruning 40% + QAT	40%	Có	INT8

Mỗi cấu hình dùng cùng dataset split và quy trình đánh giá. Tên run, seed, checkpoint nguồn và cấu hình export được lưu để Model Registry có thể truy ngược artifact về thí nghiệm tạo ra nó.

3.5.3 So sánh kết quả

So sánh tập trung vào mức thay đổi accuracy và speedup so với baseline. Cần đặc biệt kiểm tra xem pruning và QAT có cộng dồn suy giảm hay không, đồng thời đánh giá lợi ích thực trên từng thiết bị. Kết quả chi tiết được trình bày ở Chương 6; chương này mô tả quy trình và tiêu chí lựa chọn.

3.6 Chuyển đổi sang TensorRT

3.6.1 Export ONNX

Mỗi checkpoint ứng viên được export ONNX với batch và input shape [CẦN BỔ SUNG: cấu hình]. ONNX checker xác nhận graph hợp lệ; inference đối chiếu trên tập mẫu đo sai khác đầu ra và detection cuối. Chỉ graph đạt kiểm tra mới được đưa vào build engine.

3.6.2 TensorRT FP16

Engine FP16 được tạo từ baseline và các cấu hình pruning. Build log ghi TensorRT version, workspace, optimization profile, tactic và thời gian build. Sau warm-up, engine được benchmark với cùng input và cùng hậu xử lý.

3.6.3 TensorRT INT8

Engine INT8 được tạo từ mô hình QAT và pruning-QAT. Nếu pipeline cần calibration bổ sung, tập calibration phải tách khỏi test và đại diện dữ liệu thực tế. Layer precision được kiểm tra để xác định tỷ lệ toán tử thực sự chạy INT8.

3.6.4 Kiểm tra độ chính xác sau chuyển đổi

Kiểm tra gồm ba mức: sai khác tensor trên mẫu cố định, sai khác detection theo IoU/confidence và chênh lệch mAP trên toàn tập test. Ngưỡng chấp nhận được xác định trước. Artifact không đạt ngưỡng bị đánh dấu rejected và không được phép phát hành dù latency tốt.

3.7 Lựa chọn mô hình triển khai

Model triển khai được chọn từ các cấu hình không bị trội theo tập tiêu chí chất lượng và hiệu năng. Trước hết, loại các cấu hình không đạt ngưỡng accuracy, memory hoặc latency. Trong nhóm còn lại, ưu tiên cấu hình ổn định trên cả hai thiết bị và không phụ thuộc vào một tối ưu khó tái lập.

Kết quả lựa chọn cần ghi rõ model version, pruning ratio, QAT configuration, precision, input shape, checksum, engine tương ứng với từng device class và bộ chỉ số

benchmark. Model được chọn là đầu vào của Model Registry và kịch bản Deployment trong các chương tiếp theo, không phải một tệp tách rời khỏi bằng chứng thực nghiệm.

Bảng 3.4: Quyết định lựa chọn model triển khai

Thuộc tính	Giá trị
Model version	[CẦN BỔ SUNG: phiên bản được chọn]
Phương pháp tối ưu	[CẦN BỔ SUNG: baseline/pruning/QAT/pruning + QAT]
Precision	[CẦN BỔ SUNG: FP16/INT8]
Accuracy	[CẦN BỔ SUNG: Precision, Recall, mAP]
Jetson Nano	[CẦN BỔ SUNG: FPS, latency, memory, power]
Jetson Orin Nano	[CẦN BỔ SUNG: FPS, latency, memory, power]
Lý do lựa chọn	[CẦN BỔ SUNG: phân tích trade-off]

3.8 Kết chương

Chương 3 đã xây dựng quy trình từ YOLO baseline đến pruning, QAT, kết hợp hai kỹ thuật và tạo TensorRT engine. Quy trình đặt accuracy ngang hàng với latency, FPS, model size, GPU memory và power, đồng thời yêu cầu truy vết từ artifact về thí nghiệm. Chương 4 sử dụng model artifact và các yêu cầu vòng đời này để phân tích, thiết kế nền tảng Edge AI.

CHƯƠNG 4. PHÂN TÍCH VÀ THIẾT KẾ NỀN TẢNG EDGE AI

Sau khi xác định pipeline tối ưu mô hình, chương này phân tích yêu cầu và thiết kế nền tảng đưa model vào vận hành. Thiết kế tách Cloud Control Plane khỏi Edge Plane, chia domain theo DDD và đặt AI Copilot sau lớp Knowledge, tool schema, authorization và audit.

4.1 Yêu cầu hệ thống

4.1.1 Functional Requirements

Các yêu cầu chức năng được nhóm theo bounded context để mỗi chức năng có chủ sở hữu dữ liệu và quy tắc nghiệp vụ rõ ràng.

Bảng 4.1: Yêu cầu chức năng của nền tảng

Mã	Nhóm	Mô tả
FR01	Fleet Management	Đăng ký node, quản lý danh tính, capability, kết nối và trạng thái.
FR02	Model Management	Quản lý model, version, artifact, checksum, benchmark và compatibility.
FR03	Deployment	Tạo rollout, chọn target, canary/batch, theo dõi tiến độ và rollback.
FR04	Runtime Management	Khởi động, dừng, cấu hình và theo dõi vòng đời AI Runtime.
FR05	Monitoring	Thu nhận metric/log/event, truy vấn lịch sử và hiển thị dashboard.
FR06	Incident Management	Tạo, phân loại, gán người xử lý, liên kết bằng chứng và đóng incident.
FR07	Knowledge Management	Nạp, chia đoạn, lập chỉ mục và truy xuất tài liệu/runbook.
FR08	AI Copilot	Hỏi đáp có căn cứ, gọi tool đọc và lập kế hoạch hành động có phê duyệt.
FR09	Model Optimization	Lưu run tối ưu, cấu hình, chỉ số và liên kết model artifact.

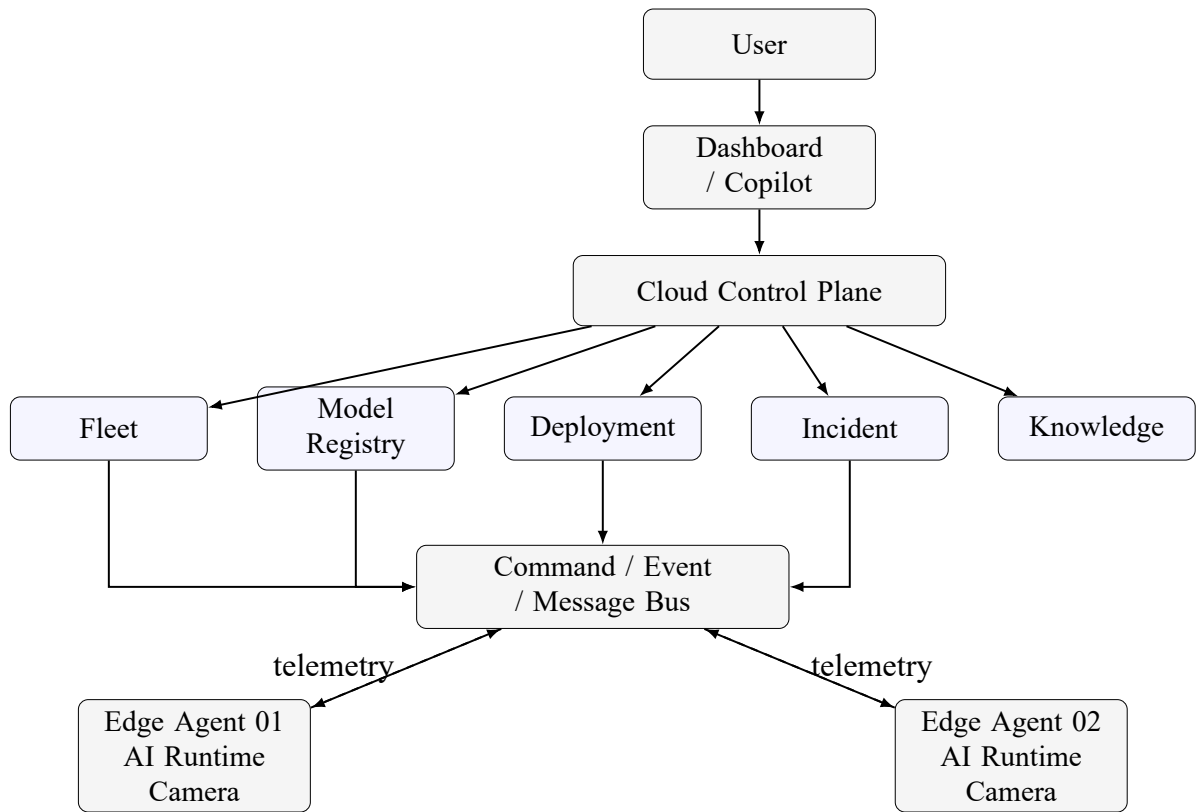
4.1.2 *Non-functional Requirements*

- **Performance:** command và telemetry có độ trễ trong ngưỡng; API và Dashboard phản hồi được ở quy mô thí nghiệm.
- **Scalability:** dịch vụ stateless có thể nhân bản; luồng telemetry và command được phân vùng theo node hoặc tenant.
- **Reliability:** Edge tiếp tục inference khi Cloud gián đoạn; command có idempotency; deployment hỗ trợ retry và rollback.
- **Security:** xác thực node/người dùng, TLS, RBAC, checksum artifact, secret management và audit log theo nguyên tắc đặc quyền tối thiểu [13].
- **Observability:** mọi dịch vụ phát metric, structured log, trace hoặc correlation id đủ để truy nguyên một deployment từ Cloud đến Edge.

Các ngưỡng định lượng cho từng yêu cầu được xác định trong Chương 6 dựa trên môi trường thực nghiệm thực tế.

4.2 Kiến trúc tổng thể

Kiến trúc tổng thể được thể hiện trong Hình 4.1. Người dùng thao tác qua Dashboard hoặc Copilot. Cloud Control Plane cung cấp các domain service và phát command qua message bus. Mỗi Edge Agent quản lý một AI Runtime gắn với camera và gửi actual state, metric, event trở lại Cloud.



Hình 4.1: Kiến trúc tổng thể nền tảng Edge AI

Control Plane là nguồn sự thật cho desired state; Edge Agent là nguồn sự thật cho actual state tức thời. Hai trạng thái không bị đồng nhất: chênh lệch giữa chúng là tín hiệu để hệ thống tiếp tục reconcile hoặc tạo incident. Dịch vụ không gọi trực tiếp vào nội bộ domain khác mà dùng API/event đã công bố.

4.3 Thiết kế Domain

4.3.1 Fleet Domain

Fleet quản lý aggregate Node với identity, device class, capability, labels, connection status và actual state. Fleet nhận heartbeat từ Edge Agent, đánh dấu offline theo timeout và cung cấp truy vấn target cho Deployment. Fleet không lưu toàn bộ time-series metric; dữ liệu này thuộc Ops.

4.3.2 Model Registry Domain

Model Registry quản lý Model, ModelVersion và Artifact. Invariant chính là một version đã phát hành không được thay đổi nội dung; checksum phải duy nhất với artifact; trạng thái chỉ chuyển từ draft qua validated đến released hoặc rejected. Benchmark và

compatibility quyết định device class nào được phép nhận artifact.

4.3.3 *Deployment Domain*

Deployment quản lý kế hoạch rollout, tập target, batch/canary, policy, tiến độ và rollback. Aggregate chỉ hoàn thành khi mọi target đạt trạng thái kết thúc phù hợp. Policy có thể dừng rollout nếu failure rate hoặc health metric vượt ngưỡng.

4.3.4 *Command Domain*

Command biểu diễn yêu cầu cụ thể gửi xuống node, gồm command id, type, payload, target, expected version, deadline và idempotency key. Vòng đời gồm queued, delivered, accepted, running, succeeded, failed, expired và cancelled. Kết quả command là bằng chứng cho Deployment hoặc thao tác Runtime.

4.3.5 *Incident Domain*

Incident quản lý sự cố từ lúc phát hiện đến khi đóng. Incident chứa severity, source, affected resources, timeline, evidence, assignee và resolution. Nhiều alert tương quan có thể gộp vào một incident để tránh tạo nhiễu cho người vận hành.

4.3.6 *Knowledge Domain*

Knowledge quản lý Document, Revision, Chunk và Indexing Job. Tài liệu phải có nguồn, phiên bản và quyền truy cập. Kết quả search trả về citation gồm document id, revision và đoạn liên quan để Copilot có thể giải thích căn cứ.

4.3.7 *Ops Domain*

Ops tiếp nhận telemetry, chuẩn hóa metric name, lưu time series và đánh giá rule. Ops cũng cung cấp truy vấn runtime metrics cho Dashboard và Copilot. Dữ liệu có retention policy để kiểm soát chi phí lưu trữ.

4.3.8 *Copilot*

Copilot quản lý Conversation, Message, ToolCall và Approval. Context hội thoại không thay thế dữ liệu domain; khi cần trạng thái mới nhất, Agent phải gọi tool. Mọi tool call lưu actor, tham số, kết quả, latency và quyết định policy.

4.4 Thiết kế Cloud Control Plane

4.4.1 *Fleet Service*

Fleet Service cung cấp API đăng ký, tra cứu node, cập nhật label và capability. Heartbeat consumer cập nhật last-seen và actual state theo sequence number để không ghi đè dữ liệu mới bằng bản tin cũ. Node identity được liên kết với thông tin xác thực của kết nối Edge.

4.4.2 *Model Registry*

Model Registry Service phát hành presigned URL hoặc cơ chế tải tương đương cho artifact. Khi upload hoàn tất, dịch vụ kiểm tra checksum, metadata và trạng thái validation. Chỉ artifact released và tương thích target mới được Deployment tham chiếu.

4.4.3 *Deployment Service*

Deployment Service tạo target snapshot, chia batch, phát command và tổng hợp kết quả. State machine của rollout được lưu bền vững để dịch vụ có thể tiếp tục sau khi khởi động lại. Scheduler không gửi batch mới khi batch hiện tại chưa đạt điều kiện thành công.

4.4.4 *Command Service*

Command Service ghi command trước khi publish, theo dõi acknowledgement và áp dụng timeout/retry. Retry giữ nguyên idempotency key để Edge Agent nhận biết lệnh đã thực thi. Kết quả được phát thành event cho Deployment, Audit và Incident.

4.4.5 *Incident Service*

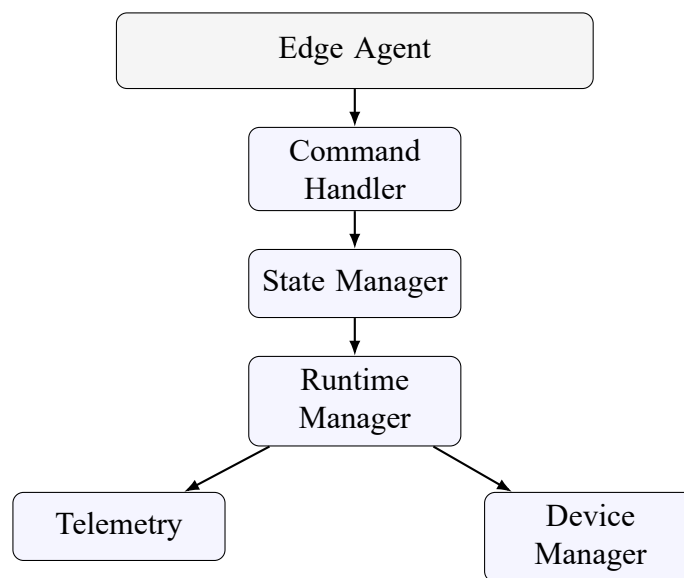
Incident Service nhận detection event, áp dụng deduplication/correlation và tạo incident. Service quản lý timeline, comment, assignment, acknowledgement và resolution. Copilot chỉ đề xuất thao tác; thay đổi severity hoặc đóng incident vẫn đi qua authorization của Incident domain.

4.4.6 Knowledge Service

Knowledge Service xử lý ingestion, chunking, embedding, keyword index và hybrid search. API search() trả danh sách ứng viên ngắn; API get() trả nội dung đầy đủ theo quyền. Tách hai bước giúp Agent chọn tài liệu trước khi đưa nội dung lớn vào context.

4.5 Thiết kế Edge Agent

Edge Agent được chia thành năm thành phần như Hình 4.2.



Hình 4.2: Các thành phần chính của Edge Agent

4.5.1 Command Processing

Command Handler kiểm tra chữ ký hoặc danh tính nguồn, schema, deadline, expected state và idempotency key. Lệnh hợp lệ được ghi vào local store trước khi thực thi. Agent gửi acknowledgement sớm, sau đó báo running và kết quả cuối với reason code chi tiết.

4.5.2 Desired State và Actual State

State Manager duy trì desired state nhận từ Cloud và actual state quan sát tại node. Reconcile loop so sánh hai trạng thái, sinh hành động nhỏ và có thể lặp. Version giúp bỏ qua desired state cũ đến muộn.

4.5.3 *Model Lifecycle*

Model lifecycle gồm download, verify, stage, validate, activate, retain và remove. Artifact được tải vào thư mục tạm, kiểm tra checksum và compatibility trước khi đổi con trỏ active. Phiên bản trước được giữ đến khi model mới vượt health check.

4.5.4 *Runtime Lifecycle*

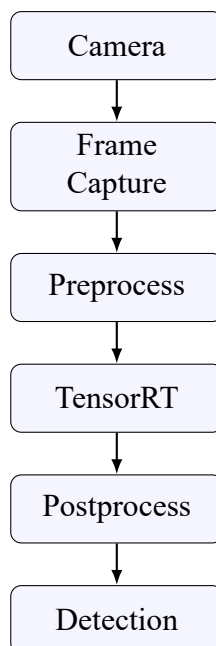
Runtime Manager tạo process/container, truyền cấu hình, theo dõi health và thu exit code. Start/stop/restart là thao tác idempotent. Khi runtime lỗi lặp lại, Agent giới hạn restart để tránh vòng lặp gây quá tải và phát event cho Incident.

4.5.5 *Telemetry*

Telemetry Collector thu metric thiết bị và runtime theo chu kỳ. Dữ liệu được gắn timestamp, node id, runtime id, model version và sequence number. Bộ đệm ngoại tuyến có giới hạn dung lượng, ưu tiên event/incident hơn metric độ phân giải cao.

4.6 Thiết kế Edge Runtime

Luồng xử lý của Edge Runtime được thể hiện trong Hình 4.3.



Hình 4.3: Thiết kế pipeline Edge Runtime

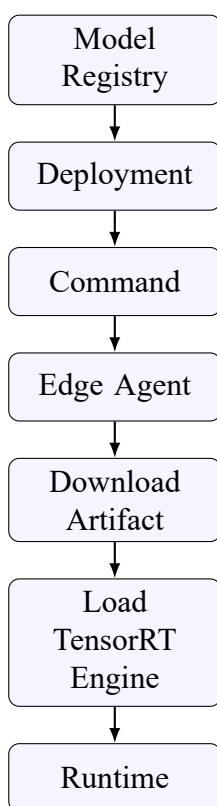
Các stage giao tiếp qua hàng đợi có kích thước giới hạn để tránh tăng latency vô

hạn. Khi consumer chậm, chính sách có thể bỏ frame cũ thay vì xử lý mọi frame. Bộ đệm GPU được cấp phát khi load model và tái sử dụng trong suốt vòng đời runtime. Metrics timer bao quanh từng stage và toàn pipeline.

Runtime nhận một cấu hình bất biến cho mỗi phiên chạy, gồm camera source, model artifact, input shape, threshold, NMS và output sink. Thay đổi cấu hình tạo phiên bản mới và được Edge Agent áp dụng có kiểm soát.

4.7 Thiết kế Model Deployment

Luồng triển khai được mô tả trong Hình 4.4.



Hình 4.4: Luồng triển khai model từ Registry đến Runtime

Trước rollout, Deployment kiểm tra trạng thái released, compatibility, target và quyền người dùng. Edge Agent thực hiện stage-and-switch: tải model mới trong khi runtime cũ vẫn chạy, xác minh engine, sau đó chuyển active version. Health check sau activation quyết định commit hoặc rollback.

Canary deployment chọn một nhóm node đại diện cho từng device class. Sau cửa sổ quan sát, hệ thống so sánh error rate, FPS, latency, nhiệt độ và incident với baseline. Rollout chỉ tiếp tục khi mọi guardrail đạt yêu cầu.

4.8 Thiết kế Monitoring và Incident

Hệ thống giám sát (Monitoring) và phát hiện sự cố (Incident Detection) được thiết kế nhằm theo dõi toàn diện trạng thái hoạt động của từng Edge Node từ mức phần cứng, tiến trình thực thi đến chất lượng suy luận của mô hình AI.

4.8.1 Không gian đặc trưng Telemetry (8 Telemetry Features)

Thay vì chỉ giám sát tài nguyên máy chủ thông thường, nền tảng thu thập bộ 8 đặc trưng telemetry cốt lõi thuộc ba nhóm thực thể tại Edge Node: phần cứng (*Hardware*), môi trường thực thi (*Runtime*) và chất lượng đầu ra mô hình (*AI Output*), như mô tả trong Bảng 4.2.

Bảng 4.2: Bộ 8 đặc trưng telemetry chính thu thập từ Edge Node

STT	Feature	Đơn vị	Đại diện cho
1	temperature_c	°C	Trạng thái nhiệt độ / Phần cứng (<i>Thermal/Hardware</i>)
2	inference_fps	FPS	Thông lượng xử lý (<i>Throughput</i>)
3	inference_latency_ms	ms	Độ trễ suy luận (<i>Inference performance</i>)
4	gpu_usage_pct	%	Áp lực tính toán GPU (<i>GPU pressure</i>)
5	cpu_usage_pct	%	Tải tính toán CPU (<i>CPU pressure</i>)
6	memory_usage_pct	%	Áp lực bộ nhớ RAM (<i>Memory pressure</i>)
7	queue_size	gói/khung	Ứ đọng hàng đợi pipeline (<i>Pipeline backlog</i>)
8	confidence_mean	0 – 1	Chất lượng đầu ra mô hình (<i>Model output quality</i>)

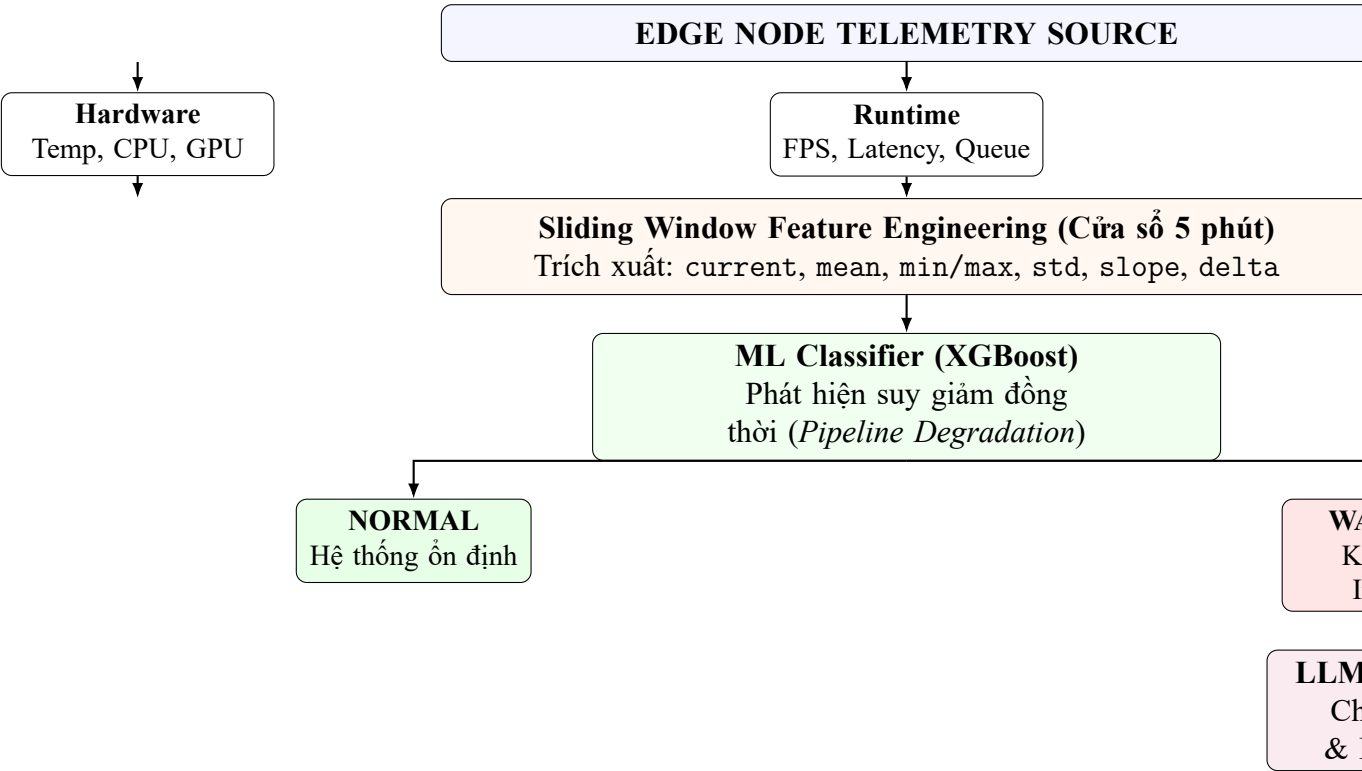
Ý nghĩa đặc biệt của confidence_mean:

Trong khi 7 đặc trưng đầu phản ánh hiệu năng và tải hệ thống/runtime, confidence_mean là telemetry đại diện trực tiếp cho chất lượng suy luận của mô hình AI. Việc tích hợp đặc trưng này mang lại ý nghĩa quan trọng cho bài toán Edge AI:

- Hệ thống có khả năng phát hiện hiện tượng suy giảm chất lượng dữ liệu đầu vào hoặc thoái hóa mô hình (*input/model degradation*) ngay cả khi các chỉ số phần cứng (CPU, GPU, nhiệt độ) và hiệu năng (FPS, Latency) vẫn hoạt động ở mức hoàn toàn bình thường.
- Giúp phát hiện sớm hiện tượng camera bị mờ, góc quay lệch, ánh sáng thay đổi đột ngột hoặc camera bị che khuất mà các giải pháp giám sát hạ tầng truyền thống hoàn toàn bỏ sót.

4.8.2 Kiến trúc phát hiện sự cố đa tầng và Feature Engineering

Luồng thu thập, xử lý chuỗi thời gian và phát hiện sự cố được minh họa trong Hình 4.5.



Hình 4.5: Kiến trúc xử lý đặc trưng Telemetry và phân loại sự cố kết hợp LLM Agent

Để mô hình học máy phân loại chính xác mà không bị nhiễu bởi các biến động tức thời, hệ thống không huấn luyện trực tiếp trên 8 giá trị đo tức thời đơn lẻ. Thay vào đó, áp dụng kỹ thuật **Sliding Window 5 phút** để trích xuất các đặc trưng thời gian (*Temporal Features*):

- **Giá trị tức thời & trung bình:** current, mean.
- **Khoảng biến thiên & độ phân tán:** min, max, std (độ lệch chuẩn).
- **Xu hướng biến thiên:** slope (độ dốc hồi quy), delta (mức thay đổi so với đầu cửa sổ).

4.8.3 Cơ chế phát hiện suy giảm đồng thời (Pipeline Degradation)

Điểm ưu việt của mô hình XGBoost phân loại trạng thái nằm ở khả năng học quan hệ tương quan phi tuyến giữa các chiều đặc trưng, phát hiện sự suy thoái toàn diện của pipeline Edge AI:

- **Trường hợp suy thoái hệ thống (WARNING):** Khi GPU ↑, Temperature ↑,

Latency ↑, Queue ↑ trong khi FPS ↓ và Confidence ↓↓. Dù từng chỉ số có thể chưa chạm ngưỡng ngắt cứng (hard threshold), mô hình vẫn kịp thời dự đoán cảnh báo nguy cơ tắc nghẽn hoặc nghẽn cổ chai cục bộ.

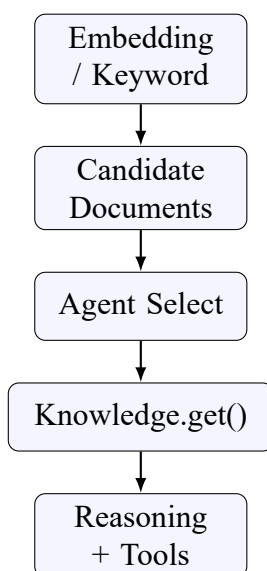
- **Trường hợp tải tăng bình thường (NORMAL):** Khi GPU ↑ do có đối tượng xuất hiện liên tục trong khung hình, nhưng Temperature, Latency, FPS, Queue và Confidence vẫn duy trì ổn định → Mô hình xác định là trạng thái hoạt động bình thường, loại bỏ triệt để cảnh báo sai (*false positives*).

Khi mô hình dự báo trạng thái **WARNING**, hệ thống sẽ tạo một bản ghi *Incident*, tổng hợp snapshot metric, lịch sử lệnh và chuyển tiếp cho **LLM AI Agent** để thực hiện chẩn đoán nguyên nhân gốc và đề xuất giải pháp xử lý (*Runbook*).

4.9 Thiết kế Knowledge và Copilot

Copilot sử dụng hai nhóm nguồn: Knowledge cho tài liệu tương đối ổn định và tool cho trạng thái vận hành mới nhất. Các công cụ cốt lõi gồm:

- `Knowledge.search()` và `Knowledge.get()`;
- `Fleet.get_node()` và `Runtime.get_metrics()`;
- `Deployment.get_history()` và `Incident.get()`.



Hình 4.6: Luồng truy xuất tri thức của Copilot

Đối với yêu cầu đọc, Agent có thể thực thi tool sau khi kiểm tra quyền. Đối với yêu cầu thay đổi trạng thái, Agent tạo một kế hoạch chứa mục tiêu, target, hành động, rủi ro và điều kiện rollback. Policy engine quyết định kế hoạch có cần phê duyệt hay bị từ chối. Approval gắn với nội dung kế hoạch; nếu tham số thay đổi, phê duyệt cũ không

còn hiệu lực.

4.10 Kết chương

Chương 4 đã chuyển yêu cầu của đề tài thành kiến trúc Cloud–Edge và các bounded context rõ ràng. Thiết kế liên kết Model Registry với Deployment, Edge Agent, Runtime, Monitoring, Incident, Knowledge và Copilot; đồng thời đưa retry, idempotency, compatibility, rollback, authorization và audit vào các luồng chính. Chương 5 trình bày cách hiện thực các thành phần này.

CHƯƠNG 5. XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG

Chương này mô tả cách hiện thực kiến trúc ở Chương 4 thành các dịch vụ Cloud, Edge Agent, Edge Runtime, Monitoring, giao diện quản lý và AI Copilot. Những tên framework, phiên bản và cấu hình chưa được xác nhận từ mã nguồn được đánh dấu để bổ sung trước khi nộp báo cáo.

5.1 Môi trường triển khai

5.1.1 *Cloud Server*

Cloud Server chạy các dịch vụ Control Plane, cơ sở dữ liệu, message broker, object storage và giao diện. Môi trường thực tế gồm [CẦN BỔ SUNG: CPU, RAM, hệ điều hành], được đóng gói bằng [CẦN BỔ SUNG: Docker Compose/Kubernetes/cách triển khai thực tế]. Các phiên bản runtime, database và broker được khóa trong tệp cấu hình để môi trường có thể tái tạo.

5.1.2 *Jetson Nano*

Jetson Nano được cài [CẦN BỔ SUNG: JetPack], CUDA [CẦN BỔ SUNG: phiên bản], TensorRT [CẦN BỔ SUNG: phiên bản] và Edge Agent. Agent chạy dưới tài khoản dịch vụ, có quyền giới hạn với thư mục model, runtime socket và thiết bị camera. Power mode, clock và cấu hình quạt được cố định cho mỗi bài đo.

5.1.3 *Jetson Orin Nano*

Jetson Orin Nano sử dụng image hệ thống và dependency tương ứng với thiết bị. Mặc dù API của Agent và Runtime giống Jetson Nano, TensorRT engine được build riêng theo device class. Capability report cho biết compute capability, RAM, TensorRT version và precision hỗ trợ.

5.1.4 *Network*

Cloud và Edge kết nối qua [CẦN BỔ SUNG: LAN/VPN/Internet thực tế]. MQTT hoặc message broker dùng TLS; artifact tải qua HTTPS hoặc object storage URL có thời

hạn. Bài kiểm thử mạng mô phỏng mất kết nối, trễ và giới hạn băng thông để xác nhận cơ chế retry, offline buffer và resume download.

Bảng 5.1: Các thành phần phần mềm của môi trường triển khai

Thành phần	Công nghệ và phiên bản
Backend/API	[CẦN BỔ SUNG: framework, phiên bản]
Database	[CẦN BỔ SUNG: hệ quản trị và phiên bản]
Message broker	[CẦN BỔ SUNG: broker và phiên bản]
Object storage	[CẦN BỔ SUNG: công nghệ]
Frontend	[CẦN BỔ SUNG: framework và phiên bản]
AI Copilot/LLM	[CẦN BỔ SUNG: mô hình, runtime, cấu hình]
Edge Agent	[CẦN BỔ SUNG: ngôn ngữ, runtime]
Edge Runtime	[CẦN BỔ SUNG: ngôn ngữ, TensorRT API]

5.2 Xây dựng Cloud

5.2.1 Fleet Management

Fleet Management hiện thực API tạo và tra cứu node, endpoint nhận heartbeat và worker đánh dấu offline. Khi đăng ký, node nhận định danh và credential theo [CẦN BỔ SUNG: cơ chế thực tế]. Heartbeat cập nhật last-seen, capability, runtime summary và model version nhưng không làm mất lịch sử trạng thái.

Các thao tác thay đổi label hoặc vô hiệu hóa node phát domain event để Deployment cập nhật khả năng chọn target. Danh sách node hỗ trợ lọc theo device class, online status, label và model version.

5.2.2 Model Registry

Model Registry cung cấp luồng tạo model version, upload artifact, xác nhận checksum, ghi benchmark và release. Binary được lưu trong [CẦN BỔ SUNG: object storage], metadata lưu trong [CẦN BỔ SUNG: database]. Service từ chối release khi thiếu manifest, checksum hoặc compatibility.

Manifest tối thiểu chứa model name/version, precision, input shape, label map, preprocess, postprocess, runtime requirement và danh sách artifact theo device class. API download không trả đường dẫn lưu trữ nội bộ mà sinh URL có hạn hoặc stream qua service.

5.2.3 *Deployment Management*

Deployment Management nhận model version, selector, strategy và guardrail. Khi khởi tạo, service chụp target snapshot để rollout không thay đổi ngoài ý muốn nếu label node bị sửa giữa chừng. Scheduler tạo command theo batch và chờ kết quả trước khi tiến tới batch tiếp theo.

State machine của deployment và từng target được lưu bền vững. Worker khởi động lại có thể tiếp tục từ trạng thái trước đó. Rollback tạo command trở về last known good version, không sửa ngược lịch sử deployment.

5.2.4 *Command Management*

Command Management lưu command trong database và publish theo mẫu outbox để tránh trạng thái đã ghi nhưng chưa gửi. Consumer nhận acknowledgement/result và cập nhật trạng thái theo thứ tự hợp lệ. Mỗi command có correlation id liên kết deployment, người yêu cầu hoặc Copilot plan.

Các loại lệnh chính gồm deploy model, update configuration, start runtime, stop runtime, restart runtime và collect diagnostic. Payload được kiểm tra bằng schema riêng cho từng loại lệnh.

5.2.5 *Incident Management*

Incident Management tiếp nhận detection event từ Monitoring, chống trùng theo fingerprint và tạo timeline. API hỗ trợ acknowledge, assign, add note, change severity, resolve và reopen. Mọi thay đổi lưu actor và thời điểm để phục vụ audit.

Incident view tổng hợp affected node, runtime, model version, deployment gần nhất, metric và command. Dữ liệu tổng hợp được lấy qua query service thay vì sao chép toàn bộ sang Incident aggregate.

5.2.6 *Knowledge Management*

Knowledge Management hỗ trợ upload tài liệu, tạo revision, chia chunk, embedding và keyword indexing. Pipeline ingestion lưu trạng thái để retry từng bước. Chunk được gắn metadata như loại tài liệu, model/device liên quan, ngôn ngữ và quyền.

Hybrid search thực hiện keyword search và vector search, sau đó hợp nhất kết quả theo [CẦN BỔ SUNG: thuật toán/reranker thực tế]. Kết quả trả snippet, score, document

id và revision để Copilot tạo citation nội bộ.

5.3 Xây dựng Edge Agent

5.3.1 *MQTT Communication*

Edge Agent kết nối broker bằng MQTT qua TLS, subscribe topic command riêng của node và publish acknowledgement, result, heartbeat, event. Client id gắn với node identity; quyền topic giới hạn để node không đọc lệnh hoặc giả mạo telemetry của node khác.

Agent dùng backoff có jitter khi kết nối lại. Session và QoS được cấu hình theo mức đảm bảo cần thiết; idempotency tại ứng dụng vẫn bắt buộc vì broker có thể giao lại message.

5.3.2 *Command Handler*

Command Handler giải mã message, kiểm tra schema, deadline và command id trong local store. Lệnh mới được ghi trạng thái accepted trước khi chuyển cho handler chuyên biệt. Kết quả gồm status, reason code, message và output có cấu trúc.

Handler không chạy thao tác dài trên luồng nhận MQTT. Công việc được đưa vào worker queue có giới hạn; các lệnh xung đột trên cùng runtime hoặc model được serialize để tránh race condition.

5.3.3 *Model Downloader*

Model Downloader tải artifact vào vùng staging, hỗ trợ timeout, retry và [CẦN BỔ SUNG: resume nếu đã hiện thực]. Sau khi tải, module kiểm tra kích thước, SHA-256, manifest và dung lượng đĩa. Tập không hợp lệ bị xóa khỏi staging và không ảnh hưởng model đang active.

5.3.4 *Runtime Manager*

Runtime Manager xây dựng command line hoặc container spec từ cấu hình đã validate, khởi động tiến trình và theo dõi health endpoint/IPC. PID, runtime id, model version và thời điểm start được ghi vào local state. Khi Agent khởi động lại, module reconcile local state với tiến trình thực tế.

5.3.5 *Telemetry Collector*

Telemetry Collector đọc chỉ số hệ thống từ [CẦN BỔ SUNG: nguồn như tegrastats/system API] và metric do Runtime phát. Dữ liệu được gom batch để giảm overhead. Khi mất mạng, event và state report được ghi vào queue bền vững; metric cũ có thể được downsample hoặc loại bỏ theo chính sách dung lượng.

5.4 Xây dựng Edge Runtime

5.4.1 *Camera Input*

Camera Input hỗ trợ [CẦN BỔ SUNG: USB/CSI/RTSP/video file thực tế]. Module cấu hình độ phân giải, FPS, timeout và chính sách reconnect. Timestamp được gắn khi frame đi vào pipeline để đo end-to-end latency. Hàng đợi capture có giới hạn và ưu tiên frame mới trong tác vụ thời gian thực.

5.4.2 *YOLO Inference*

Runtime đọc manifest để tạo preprocessing đúng với model: letterbox, đổi thứ tự kênh, scale và layout tensor. Input/output buffer được cấp phát một lần và tái sử dụng. Warm-up chạy trước khi runtime chuyển sang trạng thái ready.

5.4.3 *TensorRT*

TensorRT module deserialize engine, tạo execution context và CUDA stream. Module kiểm tra input binding, shape và data type so với manifest. Lỗi deserialize hoặc thiếu plugin trả reason code rõ ràng để Edge Agent kích hoạt rollback.

5.4.4 *Result Processing*

Result Processing giải mã output, áp dụng confidence threshold, NMS và ánh xạ label. Kết quả chuẩn hóa gồm frame id, timestamp, model version và danh sách detection. Output được gửi tới [CẦN BỔ SUNG: API/message broker/lưu trữ hoặc giao diện thực tế].

5.4.5 *Runtime Metrics*

Runtime xuất thời gian capture, preprocess, inference, postprocess, end-to-end, FPS, queue depth và frame drop. Histogram được ưu tiên cho latency thay vì chỉ trung bình. Metrics gắn model version để so sánh trước và sau deployment.

5.5 Xây dựng AI Copilot

5.5.1 *Agent Architecture*

Copilot gồm conversation API, context builder, planner, tool registry, policy check, executor và response composer. System prompt mô tả vai trò, giới hạn và quy tắc citation; dữ liệu trạng thái được lấy từ tool thay vì nhúng cố định trong prompt.

5.5.2 *Knowledge Search*

Khi câu hỏi cần tài liệu, Agent gọi `Knowledge.search` với truy vấn và bộ lọc, chọn ứng viên rồi gọi `Knowledge.get`. Câu trả lời lưu danh sách document revision đã dùng. Nếu không có bằng chứng đủ mạnh, Copilot phải nêu rõ thiếu dữ liệu thay vì suy đoán.

5.5.3 *MCP Tools*

Các domain tool được công bố qua [CẦN BỔ SUNG: MCP server/cơ chế tool registry thực tế]. Mỗi tool có tên, mô tả, JSON schema đầu vào, schema đầu ra và phạm vi quyền. Tool server ánh xạ yêu cầu sang application service, không cho mô hình truy cập thẳng database.

5.5.4 *Tool Calling*

Executor validate tham số do mô hình sinh, gắn user identity và gọi policy engine. Read-only tool có thể chạy ngay khi được phép. Write tool chỉ tạo plan; sau khi người dùng phê duyệt, executor gọi application service bằng idempotency key và lưu toàn bộ audit.

5.5.5 Conversation Memory

Conversation Memory lưu message và summary có giới hạn. Thông tin nhạy cảm được lọc trước khi đưa lại vào model. Memory không được dùng như nguồn sự thật cho trạng thái node; mọi tham chiếu như “thiết bị đó” được giải quyết thành node id, sau đó tool lấy dữ liệu mới nhất.

5.6 Xây dựng Monitoring

5.6.1 Metrics Collection

Metrics từ Cloud service và Edge Agent được nhận qua [CẦN BỔ SUNG: giao thức/collector], chuẩn hóa tên và lưu trong [CẦN BỔ SUNG: time-series database]. Nhân có cardinality cao như command id không được gắn vào mọi metric; chúng phù hợp hơn với log hoặc trace.

5.6.2 Rule-based Detection

Rule engine đánh giá cửa sổ metric theo chu kỳ. Rule gồm biểu thức, duration, severity, cooldown và fingerprint template. Ví dụ: FPS thấp hơn ngưỡng trong [CẦN BỔ SUNG: thời gian] hoặc nhiệt độ vượt ngưỡng trong nhiều mẫu liên tiếp.

5.6.3 Incident Creation

Detection event được gửi đến Incident Service cùng metric snapshot và resource reference. Fingerprint giúp cập nhật incident đang mở thay vì tạo bản ghi mới. Khi metric phục hồi đủ lâu, hệ thống đánh dấu recovered nhưng vẫn yêu cầu policy quyết định có tự động đóng incident hay không.

5.6.4 Alert Processing

Alert được định tuyến theo severity và resource label tới [CẦN BỔ SUNG: kênh thông báo thực tế]. Notification chứa incident id và liên kết Dashboard, tránh gửi toàn bộ dữ liệu nhạy cảm. Retry và trạng thái delivery được lưu để đo alert delivery latency ở Chương 6.

5.7 Giao diện quản lý

5.7.1 *Dashboard*

Dashboard tổng hợp số node online/offline, runtime health, deployment đang chạy, incident theo severity và xu hướng FPS/latency. Các thẻ số liệu đều có liên kết drill-down đến danh sách nguồn, tránh hiển thị con số không thể kiểm chứng.

5.7.2 *Fleet*

Trang Fleet hỗ trợ tìm kiếm và lọc node, xem capability, heartbeat, model đang chạy, runtime status và lịch sử command. Các hành động start/stop/restart yêu cầu quyền và hộp xác nhận nêu rõ target.

5.7.3 *Models*

Trang Models hiển thị model version, precision, artifact, checksum, benchmark và compatibility. Luồng release tách khỏi upload; người dùng phải kiểm tra validation trước khi version có thể được chọn cho deployment.

5.7.4 *Deployments*

Trang Deployments cho phép chọn model, selector, canary/batch và guardrail; sau đó hiển thị kế hoạch trước khi chạy. Màn hình chi tiết thể hiện tiến độ từng node, command và lý do thất bại, đồng thời cung cấp thao tác pause, resume hoặc rollback theo quyền.

5.7.5 *Monitoring*

Trang Monitoring hiển thị biểu đồ FPS, latency, GPU, memory, power và nhiệt độ theo node/model version. Người dùng có thể đặt cùng hai khoảng thời gian hoặc hai version để quan sát hồi quy sau triển khai.

5.7.6 *Incidents*

Trang Incidents cung cấp danh sách theo severity/status, timeline, metric snapshot, deployment liên quan và runbook. Acknowledge, assign và resolve đều được ghi audit.

Nút mở Copilot truyền incident id thay vì sao chép thủ công ngữ cảnh.

5.7.7 *Copilot*

Giao diện Copilot hiển thị câu trả lời, citation, tool call và kết quả. Khi Agent tạo kế hoạch hành động, giao diện trình bày target, bước thực hiện, rủi ro và rollback; nút phê duyệt chỉ xuất hiện khi người dùng có quyền phù hợp.

5.8 Kết chương

Chương 5 đã mô tả cách hiện thực Cloud Control Plane, Edge Agent, Edge Runtime, Monitoring, Dashboard và AI Copilot. Các thành phần sử dụng state machine, schema, checksum, idempotency và audit để giữ tính nhất quán qua kết nối phân tán. Chương 6 xây dựng phép đo để kiểm chứng cả hiệu quả tối ưu model lẫn chất lượng vận hành của nền tảng.

CHƯƠNG 6. THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này đánh giá hai phần chính của đề án: hiệu quả tối ưu YOLO và khả năng vận hành của nền tảng Edge AI. Các bảng được thiết kế sẵn nhưng không điền số giả; toàn bộ ô kết quả phải được cập nhật từ phép đo trên đúng thiết bị và cấu hình được mô tả trong mục môi trường thực nghiệm.

6.1 Môi trường thực nghiệm

Môi trường phải được ghi nhận ngay trước khi đo để tránh sai khác do cập nhật hệ thống. Ngoài thông tin trong Bảng 6.1, báo cáo cần nêu model input size, camera/video source, batch size, power mode, clock, tản nhiệt, thời gian warm-up, số vòng lặp và nhiệt độ môi trường.

Bảng 6.1: Cấu hình thiết bị thực nghiệm

Thành phần	Jetson Nano	Jetson Orin Nano
GPU	[CẦN BỔ SUNG: thông tin thực tế]	[CẦN BỔ SUNG: thông tin thực tế]
CUDA	[CẦN BỔ SUNG: phiên bản]	[CẦN BỔ SUNG: phiên bản]
RAM	[CẦN BỔ SUNG: dung lượng]	[CẦN BỔ SUNG: dung lượng]
TensorRT	[CẦN BỔ SUNG: phiên bản]	[CẦN BỔ SUNG: phiên bản]
Power Mode	[CẦN BỔ SUNG: chế độ]	[CẦN BỔ SUNG: chế độ]
OS/JetPack	[CẦN BỔ SUNG: phiên bản]	[CẦN BỔ SUNG: phiên bản]
Storage	[CẦN BỔ SUNG: loại thiết bị]	[CẦN BỔ SUNG: loại thiết bị]
Cooling	[CẦN BỔ SUNG: cấu hình]	[CẦN BỔ SUNG: cấu hình]

Mỗi phép đo hiệu năng chạy ít nhất [CẦN BỔ SUNG: số lần lặp] lần sau [CẦN BỔ SUNG: số vòng] warm-up. Báo cáo sử dụng median và các phần vị p90/p95 cho latency; FPS được đo trên cùng khoảng thời gian. Power và thermal được thu trong bài chạy đủ dài để quan sát trạng thái ổn định, không chỉ đo vài giây đầu.

6.2 Benchmark mô hình

Benchmark chất lượng được thực hiện trên cùng tập test và cùng cấu hình postprocess. Cột “Định dạng” biểu diễn FP32/FP16/INT8, còn Precision ở cuối bảng là

chỉ số chất lượng nhận diện.

Bảng 6.2: Kết quả accuracy của các cấu hình tối ưu

Model	Định dạng	Pruning	QAT	mAP@50	mAP@50:95	Precision	Recall
Baseline	FP32	0%	Không	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Pruned	FP32	20%	Không	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Pruned	FP32	30%	Không	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Pruned	FP32	40%	Không	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
QAT	INT8	0%	Có	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Pruned + QAT	INT8	20%	Có	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Pruned + QAT	INT8	30%	Có	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Pruned + QAT	INT8	40%	Có	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]

Phân tích cần nêu mức thay đổi tuyệt đối và tương đối so với baseline, lớp nào suy giảm nhiều nhất và sai số đến từ pruning hay quantization. Một cấu hình không đạt ngưỡng accuracy bị loại trước khi benchmark deployment.

6.3 Benchmark trên Jetson Nano

Jetson Nano được dùng để kiểm tra cấu hình trong điều kiện tài nguyên hạn chế. Mỗi engine chạy với cùng input, batch size và power mode. GPU memory được lấy sau warm-up; power là giá trị tổng hợp trong cửa sổ chạy ổn định.

Bảng 6.3: YOLO inference trên Jetson Nano

Model	Precision	FPS	Latency	GPU Usage	GPU Memory	Power
Baseline FP32	FP32	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
TensorRT FP16	FP16	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
QAT INT8	INT8	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Pruned + QAT	INT8	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]

Báo cáo cần bổ sung latency median/p95 và biểu đồ nhiệt độ theo thời gian. Nếu thiết bị giảm xung, kết quả trước và sau throttling phải được phân biệt.

6.4 Benchmark trên Jetson Orin Nano

Quy trình đo trên Jetson Orin Nano giữ nguyên dataset, input, batch và số vòng lặp như Jetson Nano. Engine được build cho đúng device class và TensorRT version, không sao chép engine từ Nano.

Bảng 6.4: YOLO inference trên Jetson Orin Nano

Model	Precision	FPS	Latency	GPU Usage	GPU Memory	Power
Baseline FP32	FP32	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
TensorRT FP16	FP16	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
QAT INT8	INT8	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Pruned + QAT	INT8	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]

Phân tích cần làm rõ cấu hình nào tận dụng tốt hơn kiến trúc GPU mới và liệu INT8 có tạo speedup tương xứng với mức giảm accuracy hay không.

6.5 So sánh Jetson Nano và Jetson Orin Nano

So sánh giữa hai thiết bị giúp đánh giá tính di động của model và mức mở rộng hiệu năng. Speedup được tính bằng tỷ số FPS Orin/Nano trên cùng cấu hình model; latency phải dùng cùng định nghĩa và cùng pipeline.

Bảng 6.5: So sánh hiệu năng giữa Jetson Nano và Jetson Orin Nano

Model	Nano FPS	Orin FPS	Speedup	Nano Latency	Orin Latency
FP32	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
FP16	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
INT8	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Pruned + QAT	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]

Bảng 6.5 cũng kiểm chứng việc Model Registry chọn đúng artifact theo device class. Nếu cùng model version cần hai engine khác nhau, hai artifact vẫn phải liên kết về cùng checkpoint và cấu hình tối ưu.

6.6 Benchmark Edge Runtime

Benchmark Edge Runtime đo toàn chuỗi thay vì chỉ TensorRT. Timer được đặt tại ranh giới các stage; end-to-end latency tính từ lúc nhận frame đến lúc tạo detection. Frame drop và queue depth phải được theo dõi để tránh báo cáo FPS cao trong khi latency tích lũy.

Bảng 6.6: Benchmark pipeline Edge Runtime

Pipeline	Jetson Nano	Jetson Orin Nano
Camera Capture	[CẦN BỔ SUNG: ms]	[CẦN BỔ SUNG: ms]
Preprocessing	[CẦN BỔ SUNG: ms]	[CẦN BỔ SUNG: ms]
YOLO Inference	[CẦN BỔ SUNG: ms]	[CẦN BỔ SUNG: ms]
Postprocessing	[CẦN BỔ SUNG: ms]	[CẦN BỔ SUNG: ms]
End-to-End Latency	[CẦN BỔ SUNG: ms]	[CẦN BỔ SUNG: ms]
FPS	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Frame Drop Rate	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]

Phản thảo luận cần xác định nút thắt của từng thiết bị. Nếu inference không còn

chiếm phần lớn thời gian, tối ưu tiếp model sẽ đem lại lợi ích nhỏ hơn tối ưu camera, preprocessing hoặc data movement.

6.7 Benchmark Platform

Benchmark platform trả lời liệu hệ thống có thực hiện đúng và đủ nhanh các luồng quản lý hay không. Mỗi metric cần nêu cách bắt đầu/kết thúc timer, số mẫu và điều kiện mạng.

Bảng 6.7: Benchmark Edge Agent

Metric	Jetson Nano	Jetson Orin Nano
Command Latency	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Model Download Time	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Model Load Time	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Runtime Start Time	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Runtime Stop Time	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Telemetry Rate	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
CPU Usage	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]
Memory Usage	[CẦN BỔ SUNG:]	[CẦN BỔ SUNG:]

Bảng 6.8: Benchmark Model Deployment

Metric	Kết quả
Model Upload Time	[CẦN BỔ SUNG:]
Deployment Time	[CẦN BỔ SUNG:]
Deployment Success Rate	[CẦN BỔ SUNG:]
Rollback Time	[CẦN BỔ SUNG:]
Canary Deployment Time	[CẦN BỔ SUNG:]
Model Load Time	[CẦN BỔ SUNG:]

Bảng 6.9: Benchmark Monitoring và Incident

Metric	Kết quả
Telemetry Throughput	[CẦN BỔ SUNG:]
Metric Ingestion Latency	[CẦN BỔ SUNG:]
Detection Latency	[CẦN BỔ SUNG:]
Incident Creation Latency	[CẦN BỔ SUNG:]
Alert Delivery Latency	[CẦN BỔ SUNG:]

Bảng 6.10: Benchmark Knowledge và Copilot

Metric	Kết quả
Knowledge Search Latency	[CẦN BỔ SUNG:]
Document Retrieval Accuracy	[CẦN BỔ SUNG:]
Tool Call Latency	[CẦN BỔ SUNG:]
Copilot Response Latency	[CẦN BỔ SUNG:]
Average Number of Tool Calls	[CẦN BỔ SUNG:]
Successful Task Rate	[CẦN BỔ SUNG:]
Policy Violation Rate	[CẦN BỔ SUNG:]

Nếu đủ thời gian, retrieval được so sánh theo ba cấu hình Vector Search, Hybrid Search và Agentic Retrieval trên cùng bộ câu hỏi. Đây là đánh giá bổ sung; trọng tâm vẫn là khả năng Copilot hỗ trợ vận hành chính xác, có căn cứ và đúng quyền.

6.8 Đánh giá khả năng triển khai

- Kịch bản đánh giá xuyên suốt liên kết toàn bộ các chương:
1. Huấn luyện YOLO baseline và lưu kết quả đánh giá.
 2. Thực hiện pruning, fine-tuning và QAT.
 3. Tạo TensorRT INT8 engine cho từng device class.
 4. Upload model version và artifact vào Model Registry.
 5. Tạo canary deployment tới Jetson Nano.
 6. Mở rộng deployment tới Jetson Orin Nano.
 7. Theo dõi model version, FPS, latency, GPU, power và nhiệt độ.
 8. Mô phỏng hoặc tạo điều kiện để FPS giảm dưới ngưỡng.
 9. Monitoring phát hiện và Incident Service tạo sự cố.
 10. Copilot truy vấn metric, deployment history và runbook để phân tích.
 11. Người vận hành phê duyệt rollback và xác nhận runtime phục hồi.

Một lần chạy được xem là thành công khi model được triển khai đúng target, trạng thái Cloud khớp actual state, suy giảm được phát hiện trong ngưỡng thời gian, Copilot đưa ra căn cứ phù hợp và rollback phục hồi last known good version. Báo cáo cần kèm timeline, ảnh chụp Dashboard, command history và metric trước/sau.

Bảng 6.11: Kết quả kịch bản end-to-end

Tiêu chí	Kết quả	Bằng chứng
Đăng ký và phát hành model	[CẦN BỔ SUNG: Đạt/Không đạt]	[CẦN BỔ SUNG: model version]
Triển khai hai device class	[CẦN BỔ SUNG: Đạt/Không đạt]	[CẦN BỔ SUNG: deployment id]
Phát hiện FPS suy giảm	[CẦN BỔ SUNG: Đạt/Không đạt]	[CẦN BỔ SUNG: metric/incident]
Copilot phân tích đúng nguồn	[CẦN BỔ SUNG: Đạt/Không đạt]	[CẦN BỔ SUNG: tool/citation]
Rollback thành công	[CẦN BỔ SUNG: Đạt/Không đạt]	[CẦN BỔ SUNG: command/runtime state]

6.9 Kết chương

Chương 6 đã xác lập bộ phép đo cho model, hai thiết bị Jetson, toàn bộ Edge Runtime và các dịch vụ platform. Kịch bản end-to-end nối kết tối ưu YOLO với Model Registry, Deployment, Monitoring, Incident, Copilot và rollback. Sau khi điền số liệu thật, kết quả chương này là cơ sở trực tiếp cho các kết luận ở Chương 7.

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương cuối tổng hợp kết quả của hai trục nghiên cứu: tối ưu YOLO trên thiết bị Jetson và xây dựng nền tảng vận hành AI phân tán. Các kết luận định lượng chỉ được hoàn thiện sau khi điền số liệu thực nghiệm ở Chương 6.

7.1 Kết quả đạt được

7.1.1 Kết quả tối ưu YOLO

Đồ án đã xây dựng pipeline có thể tái lập gồm baseline, structured pruning, fine-tuning, QAT, export ONNX và TensorRT. Các cấu hình được đánh giá theo cùng dataset và bộ chỉ số, giúp lựa chọn model dựa trên trade-off thay vì chỉ theo FPS. Kết quả cuối cần bổ sung: cấu hình được chọn [CẦN BỔ SUNG: tên cấu hình], mức thay đổi $mAP@50:95$ [CẦN BỔ SUNG: giá trị], speedup [CẦN BỔ SUNG: giá trị] và mức giảm kích thước [CẦN BỔ SUNG: giá trị] so với baseline.

7.1.2 Kết quả Edge Runtime

Edge Runtime đã tổ chức pipeline camera, preprocessing, TensorRT inference, postprocessing và result processing với metric theo từng stage. Runtime hỗ trợ load engine theo manifest, warm-up, health check và xuất telemetry. Kết quả cần bổ sung gồm FPS, latency đầu cuối, frame drop, GPU memory, power và độ ổn định trong bài chạy kéo dài trên từng thiết bị.

7.1.3 Kết quả Edge Platform

Nền tảng đã thiết kế và hiện thực các miền Fleet, Model Registry, Deployment, Command, Incident, Knowledge và Ops; đồng thời xây dựng Edge Agent quản lý desired state, actual state, model lifecycle và runtime lifecycle. Kịch bản triển khai, canary, monitoring, incident và rollback tạo thành vòng đời vận hành thống nhất. Các chỉ số success rate, command latency, deployment time và rollback time cần được tổng hợp từ Chương 6.

7.1.4 *Kết quả AI Copilot*

AI Copilot kết hợp Knowledge Retrieval với các công cụ Fleet, Runtime, Deployment và Incident. Thiết kế tách read-only tool khỏi hành động thay đổi trạng thái, yêu cầu lập kế hoạch, kiểm tra policy, phê duyệt và audit. Kết luận cuối cần nêu successful task rate, retrieval accuracy, latency, số tool call trung bình và kết quả kiểm tra vi phạm quyền.

7.2 Hạn chế

Phạm vi thiết bị thực nghiệm hiện giới hạn ở Jetson Nano và Jetson Orin Nano, chưa đại diện cho nhiều loại GPU, NPU hoặc CPU Edge khác. Số lượng node và quy mô deployment còn nhỏ nên chưa đánh giá đầy đủ hành vi khi hàng trăm hoặc hàng nghìn thiết bị kết nối đồng thời.

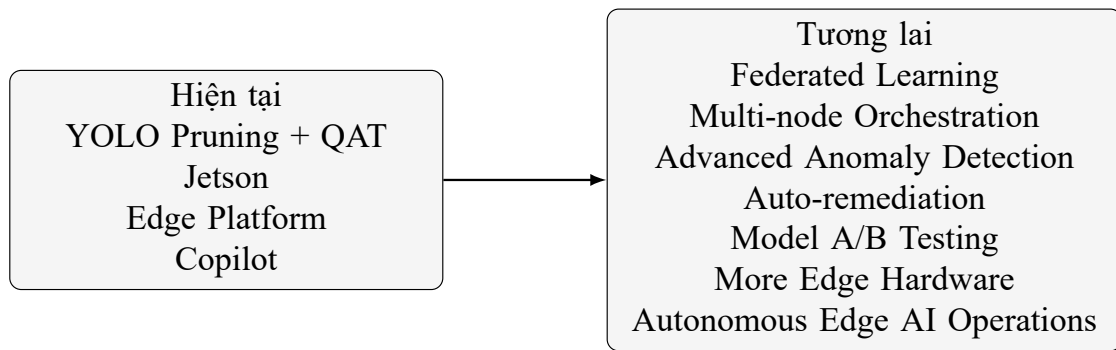
Dataset và số kịch bản môi trường có thể chưa đủ lớn để phản ánh mọi điều kiện ánh sáng, góc camera và phân bố đối tượng. Pruning ratio và cấu hình QAT mới khảo sát trong một không gian hữu hạn, chưa chứng minh là tối ưu toàn cục.

Monitoring hiện ưu tiên rule-based detection; khả năng phát hiện bất thường đa biến và drift dữ liệu chưa chuyên sâu. Copilot vẫn phụ thuộc vào chất lượng LLM, embedding, tài liệu và tool schema; cơ chế phê duyệt giảm rủi ro nhưng không loại bỏ hoàn toàn lỗi diễn giải hoặc đề xuất không phù hợp.

Nền tảng nguyên mẫu chưa thay thế hệ thống MLOps/observability quy mô lớn. Các khía cạnh như multi-region, disaster recovery, quota đa tenant, hardware root of trust và kiểm thử xâm nhập cần được nghiên cứu thêm trước khi dùng trong môi trường sản xuất.

7.3 Hướng phát triển

Hướng phát triển được xây dựng từ nền tảng hiện tại như Hình 7.1.



Hình 7.1: Các hướng phát triển từ kết quả hiện tại

Thứ nhất, nền tảng có thể mở rộng sang nhiều device class và tự động chọn model artifact theo capability, power budget và mục tiêu latency. Model A/B testing và shadow deployment giúp so sánh phiên bản mới trên dữ liệu thực trước khi chuyển toàn bộ lưu lượng.

Thứ hai, multi-node orchestration, quota và scheduler có thể tối ưu vị trí chạy runtime trên một cụm Edge. Federated learning hoặc continuous learning cho phép cải thiện model từ dữ liệu phân tán nhưng phải bổ sung quản trị dữ liệu, bảo vệ riêng tư và kiểm định model trước phát hành.

Thứ ba, Monitoring có thể phát triển từ rule-based sang anomaly detection đa biến, drift detection và dự báo tài nguyên. Incident kết hợp runbook machine-readable có thể hỗ trợ auto-remediation theo phạm vi giới hạn, rollback tự động và kiểm tra hậu điều kiện.

Cuối cùng, Copilot có thể tiến tới Autonomous Edge AI Operations theo từng cấp độ tự chủ. Mọi mức tự động hóa cao hơn phải đi kèm sandbox, policy, simulation, approval threshold, audit và cơ chế dừng khẩn cấp để bảo đảm con người vẫn kiểm soát các hành động rủi ro.

TÀI LIỆU THAM KHẢO

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.
- [2] J. Redmon and A. Farhadi, “YOLO9000: Better, Faster, Stronger,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 7263–7271.
- [3] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [4] B. Jacob *et al.*, “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.
- [5] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *International Conference on Learning Representations*, 2016.
- [6] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, “Pruning Convolutional Neural Networks for Resource Efficient Inference,” in *International Conference on Learning Representations*, 2017.
- [7] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” *arXiv preprint arXiv:1503.02531*, 2015.
- [8] ONNX Community. “Open Neural Network Exchange Documentation,” visited on Sept. 23, 2026. address: <https://onnx.ai/onnx/>
- [9] NVIDIA. “NVIDIA TensorRT Documentation,” visited on Sept. 23, 2026. address: <https://docs.nvidia.com/deeplearning/tensorrt/>
- [10] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge Computing: Vision and Challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016. DOI: 10.1109/JIOT.2016.2579198
- [11] M. Satyanarayanan, “The Emergence of Edge Computing,” *Computer*, vol. 50, no. 1, pp. 30–39, 2017. DOI: 10.1109/MC.2017.9

- [12] OASIS Open, “MQTT Version 5.0,” Organization for the Advancement of Structured Information Standards, tech. rep., 2019. visited on Sept. 23, 2026. address: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [13] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” National Institute of Standards and Technology, tech. rep. NIST Special Publication 800-207, 2020. DOI: 10.6028/NIST.SP.800-207
- [14] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [15] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.

PHỤ LỤC A. CẤU TRÚC SOURCE CODE

Phụ lục này mô tả cấu trúc mã nguồn tham chiếu. Cây thư mục cần được cập nhật theo repository thực tế trước khi nộp.

Listing A.1: Cấu trúc mã nguồn tổng quát

```
project/
|-- cloud/
|   |-- fleet/
|   |-- model_registry/
|   |-- deployment/
|   |-- command/
|   |-- incident/
|   |-- knowledge/
|   `-- copilot/
|-- edge/
|   |-- agent/
|   `-- runtime/
|-- model_optimization/
|   |-- training/
|   |-- pruning/
|   |-- qat/
|   `-- tensorrt/
|-- web/
|-- deployment/
`-- tests/
```

Mỗi module domain tách lớp lớp domain, application, infrastructure và interface nếu mã nguồn thực tế áp dụng kiến trúc này. Những thư mục chỉ mang tính minh họa phải được thay bằng đường dẫn và mô tả chính xác của dự án.

PHỤ LỤC B. DATABASE SCHEMA

Bảng B.1: Các bảng hoặc collection nghiệp vụ chính

Bảng	Khóa/quan hệ chính	Vai trò
nodes	node_id	Danh tính, capability, trạng thái và last-seen của Edge node.
models	model_id	Thông tin logic của một model.
model_versions	version_id, model_id	Phiên bản, trạng thái release và benchmark.
artifacts	artifact_id, version_id	URI, checksum, precision và compatibility.
deployments	deployment_id, version_id	Kế hoạch rollout và trạng thái tổng.
deployment_targets	deployment_id, node_id	Trạng thái triển khai trên từng node.
commands	command_id, node_id	Payload, deadline, idempotency và kết quả lệnh.
incidents	incident_id	Severity, resource, timeline và resolution.
documents	document_id	Metadata tài liệu Knowledge.
document_chunks	chunk_id, document_id	Nội dung chia đoạn và vector/index reference.
conversations	conversation_id	Phiên hội thoại Copilot.
tool_calls	tool_call_id, conversation_id	Công cụ, tham số, policy và kết quả.
audit_logs	audit_id	Actor, action, resource và thời điểm.

Sơ đồ ERD, kiểu dữ liệu, index, constraint và migration version cần được xuất từ database thực tế để bổ sung vào bản hoàn thiện.

PHỤ LỤC C. API SPECIFICATION

Bảng C.1: Nhóm API tham chiếu

Method	Endpoint	Mục đích
GET	/api/nodes	Truy vấn Fleet theo trạng thái, label và device class.
GET	/api/nodes/:id	Xem capability, actual state và model đang chạy.
POST	/api/models	Tạo model logic.
POST	/api/models/:id/versions	Tạo model version và upload session.
POST	/api/model-versions/:id/release	Phát hành phiên bản đã validation.
POST	/api/deployments	Tạo kế hoạch deployment.
POST	/api/deployments/:id/start	Bắt đầu rollout đã được phê duyệt.
POST	/api/deployments/:id/rollback	Tạo rollback cho deployment.
GET	/api/incidents	Truy vấn incident.
POST	/api/incidents/:id/resolve	Đóng incident theo quyền.
POST	/api/knowledge/search	Hybrid search tài liệu.
POST	/api/copilot/messages	Gửi message và nhận tool/citation.
POST	/api/copilot/plans/:id/approve	Phê duyệt kế hoạch hành động.

Đặc tả hoàn chỉnh cần kèm OpenAPI sinh từ mã nguồn, schema request/response, error code, authentication và ví dụ cho từng endpoint.

PHỤ LỤC D. MCP TOOL SPECIFICATION

Bảng D.1: Danh sách MCP tool tham chiếu

Tool	Mức ảnh hưởng	Chức năng
knowledge.search	Đọc	Tìm ứng viên theo truy vấn và metadata filter.
knowledge.get	Đọc	Lấy nội dung revision/chunk theo quyền.
fleet.get_node	Đọc	Lấy capability và trạng thái mới nhất của node.
runtime.get_metrics	Đọc	Truy vấn FPS, latency và tài nguyên theo cửa sổ.
deployment.get_history	Đọc	Lấy lịch sử rollout và model version.
incident.get	Đọc	Lấy timeline, evidence và trạng thái incident.
deployment.create_plan	Ghi có kiểm soát	Tạo kế hoạch deployment, chưa thực thi.
deployment.approve	Ghi, cần phê duyệt	Phê duyệt và khởi chạy kế hoạch hợp lệ.
runtime.restart_plan	Ghi có kiểm soát	Tạo kế hoạch restart runtime.

Mỗi tool phải công bố JSON schema, timeout, quyền cần thiết, idempotency behavior và error contract. Tool thay đổi trạng thái không được thiết kế thành lệnh thực thi trực tiếp nếu thiếu bước plan/approval.

PHỤ LỤC E. MODEL TRAINING CONFIGURATION

Listing E.1: Mẫu cấu hình huấn luyện và tối ưu cần lưu cùng run

```
experiment:
  name: <experiment-name>
  seed: <seed>
dataset:
  name: <dataset-name>
  version: <dataset-version>
  train_split: <path-or-id>
  val_split: <path-or-id>
  test_split: <path-or-id>
model:
  architecture: <yolo-variant>
  pretrained: <checkpoint>
  image_size: <width-height>
training:
  epochs: <epochs>
  batch_size: <batch-size>
  optimizer: <optimizer>
  learning_rate: <learning-rate>
pruning:
  method: <structured-method>
  ratios: [0.20, 0.30, 0.40]
qat:
  enabled: true
  scheme: <quantization-scheme>
export:
  onnx_opset: <opset>
  tensorrt_precisions: [fp16, int8]
```

Bản cấu hình thực tế cần bổ sung hash mã nguồn, phiên bản framework, môi trường CUDA, đường dẫn artifact và kết quả metric để bảo đảm khả năng tái lập.

PHỤ LỤC F. BENCHMARK CONFIGURATION

Bảng F.1: Cấu hình tối thiểu của một lần benchmark

Tham số	Giá trị cần lưu
Device identity	Model thiết bị, serial ẩ danh, RAM, storage
Software	OS, JetPack, CUDA, cuDNN, TensorRT, driver
Power/thermal	Power mode, clock, fan, nhiệt độ môi trường
Model	Version, checksum, precision, input shape
Input	Dataset/video/camera, độ phân giải, FPS nguồn
Runtime	Batch, queue size, threshold, NMS
Protocol	Warm-up, số vòng, thời gian chạy, số lần lặp
Metrics	Median, p90, p95, FPS, memory, GPU, power, temperature

Script benchmark phải ghi kết quả thô ở định dạng có cấu trúc và tách bước thu thập khỏi bước tạo bảng/biểu đồ. Mọi lần loại bỏ outlier cần nêu quy tắc trước, không xóa mẫu chỉ vì kết quả không thuận lợi.

PHỤ LỤC G. DEPLOYMENT CONFIGURATION

Listing G.1: Mẫu cấu hình deployment

```
deployment:
  name: <deployment-name>
  model_version: <released-version>
  target:
    device_classes: [jetson-nano, jetson-orin-nano]
    labels:
      environment: lab
  strategy:
    type: canary
    canary_size: <number-or-percent>
    batch_size: <number-or-percent>
    observation_window: <duration>
  guardrails:
    min_fps: <value>
    max_p95_latency_ms: <value>
    max_error_rate: <value>
    max_temperature_c: <value>
  rollback:
    automatic: <true-or-false>
    previous_version: <last-known-good>
```

Khi phát hành, cấu hình cần liên kết deployment id, người phê duyệt, target snapshot và model artifact theo từng device class. Secret, credential hoặc URL có token không được đưa trực tiếp vào phụ lục.